

Vibe Coding CI/CD (Foundations)

- Using Claude Code and OpenSpec
- AI overview, using AI in development, vibe coding + AI-DLC, Claude Code, simple vibe, spec-driven backend/frontend development
- Claude Code · FastAPI · Vanilla JS · Tailwind CSS · GitHub · Vercel, Railway
- Foundations
- 12 chapters: terminal basics + environment setup + skills & plugins + simple vibe + foundational development

Course Objectives

- OPENING — Course objectives
- What you will be able to do after the course — items 1–6 cover the 12 Foundations chapters, 7–8 the Advanced course
- 1. Ch. 1–3 — Understand AI, vibe coding and AI-DLC concepts
- 2. Ch. 4–5 — Terminal basics + installing, authenticating and setting up billing for Claude Code
- 3. Ch. 6 — Automate repetitive work with skills and plugins
- 4. Ch. 7 — Set up version control and collaboration with Git + GitHub
- 5. Ch. 8 — Build without design documents and feel the limits first-hand
- 6. Ch. 9 — Instruct AI systematically with CLAUDE.md and six kinds of docs
- 7. Ch. 10–11 — Complete a FastAPI backend and a Vanilla JS · Tailwind frontend
- 8. [Advanced] Extend to team development with OpenSpec and CI/CD automated deployment

Full Curriculum Roadmap

- 12 Foundations chapters + 9 Advanced chapters. One color per course, so you can see at a glance how far you have come
- **Foundations, 12 chapters — theory · environment setup · simple vibe · foundational development**
- Ch. 1 AI overview
- Ch. 2 Using AI
- Ch. 3 Vibe coding + AI-DLC
- Ch. 4 Terminal basics
- Ch. 5 Installing Claude Code
- Ch. 6 Skills & plugins
- Ch. 7 Git & GitHub
- Ch. 8 Simple vibe practice
- Ch. 9 CLAUDE.md + docs
- Ch. 10 FastAPI backend
- Ch. 11 Vanilla JS frontend
- Ch. 12 Retrospective & preview of the Advanced course
- **Advanced, 9 chapters — AI-DLC in depth · OpenSpec · CI/CD · deployment · presentation**
- Ch. 1 AI-DLC in depth
- Ch. 2 The OpenSpec methodology
- Ch. 3 Redesigning the app with OpenSpec
- Ch. 4 Tailwind CSS
- Ch. 5 Login & team features
- Ch. 6 CI/CD automated deployment
- Ch. 7 Kanban + chat
- Ch. 8 Feature expansion & debugging
- Ch. 9 Presentation & wrap-up
- What you are left with after both courses
- One full-stack app with login, teams, Kanban and chat, plus a live auto-deployed URL (Vercel + Railway)

Chapter 1. AI Overview

- **AI (Artificial Intelligence) Overview**
- The concept of AI + 7 example AI services + a taxonomy of generative AI
- What you will learn
- 1. Defining artificial intelligence
- 2. Seven example AI services
- 3. A taxonomy of generative AI services

1-1. Defining AI

- **1-1. Defining Artificial Intelligence**
- Artificial Intelligence
 - 1. Artificial intelligence — technology that imitates or substitutes for human intelligence
 - 2. Not mere numeric computation — higher-order abilities such as learning, reasoning, problem solving and perception
 - 3. Generative AI — AI that produces text, images, audio, video and code on its own
 - 4. Leading models — ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google)

1-2. Taxonomy of Generative AI Services

- **1-2. A Taxonomy of Generative AI Services — one field per slide from here on**
- Field · Representative services · Characteristics
- Text · ChatGPT, Claude, Gemini · General knowledge, conversation, code, documents
- Image · Gemini, DALL-E, Midjourney · Text to image generation
- Speech · Whisper, Clova Note, ElevenLabs · Speech recognition, transcription, synthesis
- Video · Google Veo, Grok, Sora · Cinema-grade video and audio generation
- Code (vibe) · Claude Code, Antigravity, Cursor, Codex CLI · Implements whole features autonomously from natural language
- Code (assisted) · GitHub Copilot, ChatGPT · Autocomplete, bug fixing, code review
- Local LLM · Ollama, LM Studio · Running AI offline

1-2-1. Coding (ChatGPT)

- **1-2-1. Example AI Service — Coding (ChatGPT)**
- Turning the feature you describe into code
- *(image)* A web page built with ChatGPT
- 1. Describe the feature you want and it is implemented in code
- 2. Automatically generates web pages, APIs and scripts
- 3. Ask it to fix bugs and explain code
- 4. Example prompt: "Build me a company introduction web page"
- 5. <https://chatgpt.com>
- 6. Shared hands-on link (click)

1-2-2. Vibe Coding (Claude Code / Cursor)

- **1-2-2. Example AI Service — Vibe Coding (Claude Code / Cursor)**
- The AI writes the code itself, from natural-language instructions alone
- **(image)** The Claude Code terminal screen
- 1. Claude Code — a terminal-based AI agent (the tool used in this course)
- 2. Cursor — an IDE-based AI collaborative editor
- 3. Instruct in natural language, and the AI creates, edits and runs files
- 4. You never have to write a single line of code yourself
- 5. <https://claude.ai/code> / <https://www.cursor.com>

1-2-3. Writing Documents (Claude)

- **1-2-3. Example AI Service — Document Writing (Claude)**
- Fiction, proposals, reports, slide graphics
- *(image)* Writing a document in Claude
- 1. Writing fiction and proposals
- 2. Creating slide graphics and reports
- 3. Automatically drafting emails
- 4. Organizing and summarizing meeting notes
- 5. <https://claude.ai>

1-2-4. Image Generation (Gemini)

- **1-2-4. Example AI Service — Image Generation (Gemini)**
- High-quality images from a single line of text
- **(image)** An example of image generation in Gemini
- 1. One line of text produces a high-quality image
- 2. Style transfer and editing of photos
- 3. Uses Gemini 3.1 Flash Image
- 4. Producing design drafts and marketing imagery
- 5. <https://gemini.google.com>

1-2-5. Speech Recognition (Whisper / Clova Note)

- **1-2-5. Example AI Service — Speech Recognition (Whisper / Clova Note)**
- Speech to text, and text to speech
- *(image)* The Naver Clova Note screen
- 1. OpenAI Whisper — automatic subtitles for YouTube videos
- 2. Naver Clova Note — automatic transcription of meeting and lecture recordings
- 3. Can generate subtitles in real time
- 4. Multilingual support
- 5. <https://openai.com/research/whisper> / <https://clovanote.naver.com>

1-2-6. Video Generation (Grok / Veo 3)

- **1-2-6. Example AI Service — Video Generation (Grok / Veo 3)**
- Cinema-grade video and audio generated together
-  An example of video generation with Google Veo 3
- 1. xAI Grok Imagine — creative, emotive video generation
- 2. Google Veo 3 — cinema-grade video with audio
- 3. Generates video from a single line of text
- 4. Used for advertising and content production
- 5. <https://x.ai> / <https://deepmind.google/technologies/veo>

1-2-7. Local LLMs (Ollama / LM Studio)

- **1-2-7. Example AI Service — Local LLMs (Ollama / LM Studio)**
- Running an LLM on your own machine, with no internet
-  Ollama models and the LM Studio screen
- 1. Ollama — runs and serves local LLMs from the CLI (ollama.com)
- 2. LM Studio — a GUI tool for running local LLMs
- 3. Upside: no internet needed, and no data leaves your machine
- 4. Downside: needs a powerful GPU, and performs worse than commercial models
- 5. <https://ollama.com> / <https://lmstudio.ai>

Checkpoint

- 1. Defining artificial intelligence
- 2. Seven example AI services
- 3. A taxonomy of generative AI

Chapter 2. Using AI in Development

- **Using AI in Development**
- How to use AI across the whole software development process
- What you will learn
- 1. Using AI at each stage of development
- 2. AI-assisted coding vs. vibe coding
- 3. Claude Code — the core tool of this course

2-1. The Planning Stage

- **2-1. The Planning Stage — Planning with AI**
- From the first idea through the PRD to screen design — AI tools cut planning time
- Stage · Tool · Key capability
- Research & ideation · Perplexity · Source-backed market research, competitor analysis, trend spotting
- Research & ideation · Claude / ChatGPT · Fleshing out concepts, user personas, draft functional specs
- Proposals & documentation · Gamma · Enter text and get a proposal as slides or a landing page
- Proposals & documentation · Manyfast · Turns an idea into a PRD plus wireframes in one pass
- UI/UX design · Figma AI · Prompts into wireframes and UI prototypes, fast
- UI/UX design · Uizard · Turns sketches and text into high-fidelity UI screens
- Mind mapping & structuring · EdrawMind · Visualizes service structure and core logic as a mind map

2-1-1. Perplexity

- **2-1-1. Perplexity — Source-Backed AI Research**
- Strong where you need to verify sources
-  The Perplexity screen
- 1. Search results carry their sources, so you can judge how reliable the information is
- 2. Tuned for market research, competitor analysis and trend spotting
- 3. Suggests related questions automatically, making deep research more efficient
- 4. Example prompt: "Analyze 2025 trends in the task-management SaaS market"
- 5. <https://www.perplexity.ai>

2-1-2. ChatGPT

- **2-1-2. Claude / ChatGPT — Service Planning and Spec Writing**
- From fleshing out the concept to personas and functional specs
- *(image)* Planning in Claude — a CLAUDE.md example
- 1. Flesh out the service concept — turn an idea into a written proposal
- 2. Generate user personas automatically and derive target-user scenarios
- 3. Draft a functional spec — feature list, priorities and conditions
- 4. Example prompt: "Create three core user personas for a B2B collaboration tool"
- 5. <https://claude.ai> / <https://chatgpt.com>

2-1-3. Gamma

- **2-1-3. Gamma — Automatic AI Proposals and Slide Decks**
- One line of text becomes a deck or a landing-page-style proposal
- *(image)* Slides generated automatically by Gamma
- 1. Text alone produces a proposal as a slide deck or a landing page
- 2. Headers, cards and charts come together — no design work needed
- 3. An AI assistant can add, edit and improve slides
- 4. Example prompt: "Create a service proposal for a B2B collaboration tool"
- 5. <https://gamma.app>

2-1-4. Manyfast

- **2-1-4. Manyfast — Idea to PRD (Product Requirements Document) plus Wireframes**
- From proposal to wireframe in a single pass
-  Manyfast generating a PRD and functional spec
- 1. Enter an idea and a PRD (product requirements document) is generated automatically
- 2. Wireframes come with it — planning and design advance together
- 3. The PRD covers feature definitions, priorities, constraints and success metrics
- 4. Example prompt: "Create a PRD and wireframes for a task-management app"
- 5. <https://www.manyfast.ai>

2-1-5. Figma AI

- **2-1-5. Figma AI — Prompt to UI Prototype**
- Wireframes and UI built automatically from your requirements
-  Figma AI generating a wireframe
- 1. A text prompt quickly produces wireframes and UI prototypes
- 2. It stays inside Figma, so you can edit the result straight away
- 3. Supports both mobile and web layouts
- 4. Example prompt: "Design the mobile home screen for a B2B collaboration tool"
- 5. <https://www.figma.com>

2-1-6. Uizard

- **2-1-6. Uizard — Sketches and Text to High-Fidelity UI**
- Even a hand-drawn sketch becomes a UI
-  Uizard's template picker
- 1. Turns a written description into a high-fidelity UI screen
- 2. Converts a photo of a hand-drawn sketch into a UI screen automatically
- 3. Generates three variants so you can pick the style you want
- 4. Example use: describe a screen in text, then compare three UI variants and choose
- 5. <https://uizard.io>

2-1-7. EdrawMind

- **2-1-7. EdrawMind — Mind Mapping and Structuring Logic**
- Visualizing service structure and core logic
- **(image)** A mind map visualized in EdrawMind
- 1. Visualize the structure of a service plan and its core logic as a mind map
- 2. AI generation builds the initial skeleton of the map quickly
- 3. Grasp the whole service across four axes: users, features, business and technology
- 4. Example use: build a planning mind map and share the structure with teammates
- 5. <https://www.edrawmind.com>

2-2. AI-Assisted Coding

- The AI assists while the developer writes the code
- 1. The developer writes the code; the AI's job stops at suggestions
- 2. Autocomplete suggestions, bug fixes, code review
- 3. An extension of conventional development — you still have to understand the code
- 4. Leading tools: GitHub Copilot, ChatGPT

2-2-1. GitHub Copilot

- **2-2-1. GitHub Copilot — Autocomplete as an IDE Plugin**
- A Microsoft–OpenAI collaboration, released in 2022
- *(image)* GitHub Copilot in action
- 1. Installed as an IDE plugin (VS Code, JetBrains and others)
- 2. Suggests completions in real time as you type
- 3. Press Tab to accept, or keep typing to ignore
- 4. \$10/month for individuals, \$19 for business
- 5. <https://github.com/features/copilot>

2-2-2. Bug Fixing and Refactoring

- **2-2-2. ChatGPT — Bug Fixing and Refactoring**
- The most general-purpose AI coding tool
-  ChatGPT generating HTML for a to-do app
- 1. Paste in code and ask for the bug to be fixed
- 2. Explain code, refactor it, or generate tests
- 3. You can ask it to translate code into another language
- 4. The free tier is already useful enough

2-3. AI Vibe Coding

- One sentence of natural language, and the AI writes all the code
- 1. The developer does not write the code
- 2. You convey intent to the AI, and it implements the whole feature
- 3. The developer's role is conveying intent and verifying the result
- 4. Leading tools: Claude Code, Antigravity, Cursor, OpenAI Codex CLI

2-3-1. Claude Code

- **2-3-1. Claude Code — The Core Tool of This Course**
- Built by Anthropic; a terminal-based AI agent
-  Claude Code creating index.html from a natural-language instruction
- 1. You instruct it in natural language from the terminal
- 2. It creates, edits and runs files autonomously
- 3. CLAUDE.md keeps the project context
- 4. Uses an API key (provided by the instructor) for this course
- 5. <https://claude.ai/code>

2-3-2. Antigravity

- **2-3-2. Antigravity — Google DeepMind's AI Agent**
- A next-generation AI development environment built on VS Code
-  Antigravity executing an implementation plan
- 1. Built by Google DeepMind, on top of VS Code
- 2. Runs on Gemini models, with agents executing in parallel
- 3. A browser agent operates web services directly
- 4. An alternative to Claude Code if you live in the Google ecosystem
- 5. <https://antigravity.dev>

2-3-3. Cursor

- **2-3-3. Cursor — An IDE-Based AI Collaborative Editor**
- A fork of VS Code with AI built in
-  Cursor analyzing code through @Codebase chat
- 1. Built on VS Code, so your existing setup carries over
- 2. AI edits at file level, driven through chat
- 3. You develop code in conversation with the AI
- 4. \$20/month, with a student discount available
- 5. <https://www.cursor.com>

2-3-4. OpenAI Codex CLI

- **2-3-4. OpenAI Codex CLI — Terminal-Based AI Coding**
- Same family as Claude Code — a vibe coding tool
-  Codex CLI — "Explain this code base"
- 1. A terminal-based AI agent, much like Claude Code
- 2. Natural language in, generated code out
- 3. A coding-specific CLI tool made by OpenAI
- 4. Codex CLI is a vibe coding tool, not an assisted-coding one
- 5. <https://openai.com/blog/openai-codex>

2-4. Assisted vs. Vibe Coding

- **2-4. AI-Assisted Coding vs. AI Vibe Coding**
- Aspect · AI-assisted coding · AI vibe coding
- Who writes the code · The developer · The AI generates all of it
- Developer's role · Code author · Conveying intent and verifying results
- Skills needed · Coding ability is essential · Judgment and prompting
- Error handling · The developer fixes it · The AI fixes it itself
- Speed gain · 20–30% over conventional work · 50–80% over conventional work
- Leading tools · Copilot, ChatGPT · Claude Code, Antigravity, Cursor, Codex CLI
- This course · – · Builds a complete full-stack app through vibe coding

2-5. The Service Stage — AI API Integration

- **2-5. The Service Stage — Integrating AI APIs**
- Putting AI features inside your own app
- 1. AI API integration — build chatbots, summarization, search and classification with the OpenAI, Claude and Gemini APIs
- 2. RAG — Retrieval-Augmented Generation, an AI answering system grounded in your own documents
- 3. LangChain / LangGraph / Flowise — frameworks for building AI feature pipelines
- 4. This course covers the concepts only; the depth is in a separate LangChain/Flowise course
- 5. <https://aistudio.google.com> / <https://console.anthropic.com> / <https://platform.openai.com>

2-5-1. Google AI Studio

- **2-5-1. AI API Integration — Google AI Studio**
- The easiest place to start with AI APIs
-  Gemini API usage in Google AI Studio
- 1. Google AI Studio — free Gemini API keys, testable right in the browser
- 2. Anthropic Console — issue Claude API keys and choose models such as claude-3-5-sonnet
- 3. OpenAI Platform — issue GPT-4o API keys, with code examples provided
- 4. The common flow: issue an API key, set it in your code, call the API, handle the response
- 5. Official sites: aistudio.google.com / console.anthropic.com / platform.openai.com

2-5-2. RAG

- **2-5-2. RAG — Retrieval-Augmented Generation**
- Retrieval-augmented generation — the AI reads your documents and answers from them
-  RAG vector space — semantically similar questions clustering together
- 1. A technique where the AI searches and consults external documents as it answers
- 2. The problem with plain AI — it answers nonsensically outside its training data (hallucination)
- 3. RAG's fix — store your documents in a vector database, retrieve the relevant ones per question, and let the AI answer accurately from them
- 4. Example uses — an internal-policy chatbot, product-manual Q&A, legal document search

2-5-3. LangChain

- **2-5-3. LangChain — A Framework for Building AI Apps**
- Linking several AI capabilities together like a chain
-  A LangChain chatbot — the response and the time each step took
- 1. A Python/JS framework that makes LLM-powered apps easy to build
- 2. A chain links AI steps in sequence: input, processing, output
- 3. Supports RAG, chatbots, summarization, classification and more
- 4. Example uses — a document summarizer, a customer-support AI, automated code review
- 5. <https://www.langchain.com>

2-5-4. LangGraph

- **2-5-4. LangGraph — Graph-Based AI Agents**
- Complex AI workflows with branching and loops
-  LangGraph Studio — the memory-agent graph
- 1. An extension of LangChain that composes agents as a graph (a flow diagram)
- 2. Branching, looping and parallelism — ideal for complex agents
- 3. For example, a routing agent that picks a different AI tool depending on the type of question
- 4. Example uses — multi-step research bots, autonomous task agents, complex business-process automation
- 5. <https://langchain-ai.github.io/langgraph>

2-5-5. Flowise

- **2-5-5. Flowise — A No-Code AI Pipeline Tool**
- LangChain by drag and drop
-  Flowise — building an Agentflow and testing the chatbot
- 1. A no-code tool that drives LangChain through a GUI
- 2. Drag and drop — connect blocks to compose an AI pipeline visually
- 3. Rapid prototyping — stand up RAG, chatbots and other AI services without code
- 4. Example uses — non-developers building chatbots, quick tests of AI features, MVPs
- 5. <https://flowiseai.com>

2-6. AI by Development Stage

- **2-6. Summary — Using AI at Each Development Stage**
- Stage · Approach · Leading tools
- 1) Planning · Automating requirements, proposals and API design · Claude, ChatGPT
- 2) Assisted coding · Autocomplete, bug fixes, refactoring · Copilot, ChatGPT
- 3) Vibe coding · Implementing whole features from natural language · Claude Code, Antigravity, Cursor, Codex CLI
- 4) Service · Embedding AI features in the app · OpenAI API, LangChain

Checkpoint

- 1. How to use AI at each development stage
- 2. How the four vibe coding tools differ
- 3. What AI API integration and RAG are for

Chapter 3. Vibe Coding + AI-DLC

- **The Vibe Coding Development Philosophy + the AI-DLC Concept**
- Defining vibe coding, the AI-DLC framework, and a preview of OpenSpec
- What you will learn
 - 1. What vibe coding is
 - 2. Human vs. AI roles in AI-DLC
 - 3. Three pitfalls of vibe coding

3-1. How AI Coding Tools Evolved

- **3-1. The Evolution of AI Coding Tools — from Autocomplete to Autonomous Development**
- 2022 · GitHub Copilot · IDE plugin · Line-by-line autocomplete
- 2023 · Coding with ChatGPT · Conversational · Generating and editing code
- 2023–24 · Cursor · IDE-based · AI collaborative editor
- 2025 · Claude Code · Terminal agent · Fully autonomous development
- 2025 · Antigravity · VS Code agent · Google DeepMind

3-2. Defining Vibe Coding

- Andrej Karpathy, February 2025
- 1. "Rather than writing the code yourself, you convey intent and verify the AI's output"
- 2. The developer's role shifts — from code author to supervisor and verifier of the AI
- 3. Why you need to learn it — 84% of developers use AI, with a 55% speed gain
- 4. If you don't — you lose competitiveness and the productivity gap widens

3-3. How the Developer's Role Changes

- What is different after vibe coding
- 1. Before — writing code by hand, memorizing syntax, debugging errors
- 2. After — conveying intent, verifying results, deciding direction
- 3. The new core skills — writing good prompts, and judging the output
- 4. What does not go away — system design, business understanding, quality judgment

3-4. AI-DLC

- **3-4. AI-DLC — the AI-Driven Development Life Cycle**
- AI-Driven Development Life Cycle — a methodology proposed by AWS
- 1. What AI-DLC is — a methodology that treats AI not as a mere assistant but as a core collaborator across the whole development process
- 2. Origin — published by AWS (Amazon Web Services) out of real production experience
- 3. Core principle 1 — AI-led execution with human oversight: the AI plans and generates code, the human reviews and approves
- 4. Core principle 2 — plain vibe coding generates it all at once; AI-DLC verifies stage by stage
- 5. <https://github.com/awslabs/aidlc-workflows> / <https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle>

3-5. The Five-Stage AI-DLC Process

Grid layout: five stages across, three rows (Human / AI / Tools).

- AWS AI-Driven Development Life Cycle — human and AI roles cleanly separated at every stage
- Stage 1) Requirements analysis
 - — Human: define the goal, decide the constraints
 - — AI: analyze comparable cases
 - — Tools: external tools + OpenSpec
- Stage 2) Writing the spec
 - — Human: decide the WHY, review and approve
 - — AI: propose the WHAT and HOW
 - — Tools: OpenSpec
- Stage 3) Code generation
 - — Human: set the direction, review the result
 - — AI: generate the code
 - — Tools: Claude Code + OpenSpec
- Stage 4) Verification & testing
 - — Human: final quality judgment and approval
 - — AI: automated tests and error detection
 - — Tools: pytest + Claude Code + OpenSpec
- Stage 5) Deployment & operations
 - — Human: approve and monitor
 - — AI: run CI/CD automatically
 - — Tools: Actions + Vercel + OpenSpec
- Every stage runs on OpenSpec artifacts (proposal / specs / design / tasks). The next slide shows how roles shift stage by stage

3-6. S-DLC vs AI-DLC

- **3-6. S-DLC vs. AI-DLC — the Shift in the Developer's Role**
- The same five stages as slide 3-5, now down the vertical axis — how the developer's role changes at each. The enabling tools come in the Advanced course's OpenSpec chapter
- Stage · S-DLC (traditional) · AI-DLC · Developer's role · Enabling tool
- Requirements · Interviews, documentation · AI analyzes, the human confirms · Collector becomes definer · OpenSpec > proposal.md
- Spec & design · Hand-written UML and APIs · OpenSpec generates, the human approves · Author becomes reviewer · OpenSpec > specs/{feature}/spec.md
- Implementation · Line by line · Vibe coding automates it, the human steers · Typist becomes director · Claude Code <- design.md
- Testing · Hand-written cases · The AI generates and runs them · Author becomes judge · pytest + Claude Code <- tasks.md
- Deployment · Manual · The developer pushes, the rest is automatic · Operator becomes overseer · git push + Actions + Vercel <- tasks.md
- S-DLC: the worker · AI-DLC: the decision maker
- The developer decides why, what and when; the AI executes the how
- Judgment matters more than coding skill. The four OpenSpec artifacts that make this work come in Chapter 2 of the Advanced course

3-7. The Pitfalls of Vibe Coding

- Why you need a systematic methodology
- 1. 45% security vulnerabilities — AI-generated code can carry security problems
- 2. If you cannot read the code you cannot debug it — you cannot tell when the AI is wrong
- 3. Instructions without context produce differently structured code every time
- 4. The fix — secure consistency with CLAUDE.md and OpenSpec

3-8. Adopting SDD Is Separate from Adopting AI

- **3-8. Adopting SDD Is a Separate Matter from Adopting AI**
- SDD (Spec-Driven Development) is not a tool — it is the standardization of the development process
- Swap the AI out and the spec remains as an asset
- Claude Code · Codex · Gemini — all replaceable
- > Requirements / design intent / approval history
- **Adopting AI without SDD**
 - – Every developer prompts differently
 - – The same request produces wildly different output
 - – The reasoning behind the design is never recorded
 - – Review criteria vary from person to person
 - – There is no reference document to hand a new joiner
- **Adopting SDD**
 - – Implementation is directed against the spec — standardized
 - – Code review criteria are fixed in a document
 - – Decisions leave a record, so context can be traced
 - – Change the AI and the spec is still reusable
 - – A new joiner can be onboarded with the spec alone

3-9. Practical Principles of Vibe Coding

- Convey intent, verify the result, repeat
- 1. Be specific — spell out the stack, the structure and the features
- 2. Verify the result — run and check the AI's code too
- 3. Ask when unsure — "How does this code work?"
- 4. Instruct in small units — one thing at a time

Checkpoint

- 1. What vibe coding is
- 2. How AI-DLC separates human and AI roles
- 3. Three pitfalls of vibe coding

Chapter 4. Terminal Basics

- **Getting Your Hands Used to the Terminal with Ten Commands**
- Switching drives, creating folders, handling files, deleting, Tab completion
- What you will learn
 - 1. Type the ten commands yourself
 - 2. Tab completion and arrow-key history
 - 3. Handling the five most common errors

4-1. Why You Need the CLI

- **4-1. Why You Need to Learn the CLI (Command Line Interface)**
- Not a warm-up for one particular tool — it is the shared interface across backend, DevOps and systems roles
- **Servers have no GUI** — a Linux server with no graphical interface installed is the standard configuration. Connect over SSH and all you get is a shell
- **The operations toolchain is CLI-based** — docker, kubectl, systemctl, nginx and the rest are designed around command-level control
- **Automating repetitive work** — GUI actions cannot be reused, but commands can be bundled into scripts, re-run and scheduled
- **Faster incident response** — combine tail and grep to filter straight to the cause in a huge log
- **What a CI/CD pipeline actually is** — a CI/CD (continuous integration / delivery) workflow is a sequence of commands to run
- **What you learn keeps its value** — GUIs differ per product and get redesigned often, while POSIX standard commands keep their interface for decades
- By role
- Backend developer — starting servers, running DB migrations and tests
- DevOps engineer — builds, deployments, IaC (infrastructure as code)
- Systems engineer — checking accounts, permissions, processes and networking
- Data engineer — running batches, checking logs, registering schedules
- The CLI is not an accessory to one tool — it is the standard interface used right across development roles

4-2. Why Claude Code Is CLI-Based

- **4-2. Why Claude Code Is CLI-Based — What Only the Terminal Makes Possible**
- Unlike tools bound to an editor process, it can do the full range of what a developer does
- 1. The whole project is in scope — it explores and references the entire repository, not just the files open in an editor
- 2. It manipulates files directly — not just suggesting code but creating, editing and deleting
- 3. It runs commands and reads the output — pip install, pytest, git commit, starting a server
- 4. It behaves identically on remote machines — runs as-is on a server reached over SSH, no GUI needed
- 5. It slots into automation — `claude -p` lets you embed it in scripts and CI pipelines for unattended runs
- 6. It is independent of OS and editor — identical on macOS, Windows and Linux, whatever editor you use
- An IDE plugin: the developer leads and the AI stops at suggestions. The terminal (Claude Code): the AI executes and the developer reviews and approves

4-3. Launching PowerShell

- **4-3. Launching PowerShell — the Starting Point for Every Exercise**
- Two ways. The search box is simpler, so use that one first
- 1. Method 1 — type pow into the taskbar search box *(best)*
- 2. Pick "Windows PowerShell" at the top of the list and press Enter
- 3. Method 2 — Windows key + R, then powershell, then Enter
- 4. When you see PS C:\Users\yourname> it is running
- 5. That PS . . . > part is the prompt
- 6. The prompt tells you which folder you are currently working in
- 7. This whole course assumes PowerShell
- If the title bar says "Command Prompt" you are in CMD — close it and start again
- The bottom line is the prompt — type your commands there

4-4. Ten Essential PowerShell Commands

Command	What it does	In CMD
pwd	Which folder am I in right now	cd (typed alone)
cd 폴더명	Go into that folder	cd foldername
cd ..	Go up one level	cd ..
D:	Switch to the D drive	D:
dir	List files in the current folder	dir
mkdir 폴더명	Create a new folder	md foldername
cat 파일명	Show a file's contents	type filename
Remove-Item 파일명	Delete a single file	del filename
Remove-Item -Recurse -Force 폴더명	Delete a whole folder (cannot be undone)	rd /s /q foldername
cls	Clear the screen (files are untouched)	cls

4-5. Exercise 1 — Switch Drive and Create a Folder

- **4-5. Exercise 1 — Move to the D Drive and Create a Folder**
- White text is what you type; grey text is what the screen prints back
- Doing it in the terminal
- # 1. Move to the D drive (include the colon)
- PS C:\Users\student> D: → PS D:\>
- # 2. Check where you are → pwd → Path ---- D:\
- # 3. Create a practice folder and go into it
- PS D:\> mkdir cli-test / PS D:\> cd cli-test / PS D:\cli-test>
- Doing the same thing in File Explorer: right-click an empty spot > New > Folder
- Same outcome. Clicking into the D drive in Explorer and right-clicking to make a folder is one line: `mkdir cli-test`

4-6. Exercise 2 — Create a File, Add Content, Check It

- **4-6. Exercise 2 — Create a File, Put Content In, Check It**
- What you used to do in Notepad, in three commands
- # 1. Create an empty file → `New-Item -ItemType File memo.txt`
- Output: Directory: D:\cli-test / Mode LastWriteTime Length Name / -a--- 2026-08-05 17:27 0 memo.txt
- # 2. Write content to the file → `Set-Content memo.txt "first line"`
- # 3. Show the file's contents → `cat memo.txt` → first line
- The same thing in Explorer: right-click > New > Text Document, open it in Notepad, type and save
- `cat` dumps the contents to the screen without opening the file — far quicker than opening files one by one in Notepad

4-7. Tab Completion and Command History

- Type less and you make fewer typos; fewer typos means fewer interruptions
- 1. Type just `cli` → 2. Press Tab once and it completes → 3. Press `↑` to recall the previous command
- PS `D:\> cd cli` + Tab → PS `D:\> cd cli-test` (the rest fills itself in)
- Tab — completes folder and file names; type the first few characters and press Tab
- Tab repeatedly — if there are several candidates, keep pressing to cycle through them
- `↑` — recalls the command you just typed, so a typo does not mean retyping
- `↓` — moves forward through history
- `Ctrl + C` — force-stops a command that has hung
- `cls` — tidies the screen only; files are untouched
- Most stalled exercises come down to typing errors — Tab and the up arrow prevent nearly all of them

4-8. Copy and Paste in PowerShell

- **4-8. Copy and Paste — How PowerShell Behaves**
- Long prompts cannot realistically be typed out, so this is a prerequisite for the exercises
- **Pasting into PowerShell (pick one of three)**
- Right-click — right button inside the window; the most reliable
- Ctrl + V — the default since Windows 10
- Shift + Insert — for older consoles where the two above fail
- **Copying out of PowerShell**
- Drag then Enter — select the text, then press Enter
- Drag then right-click — same result
- Ctrl + C — copies only while text is selected
- Ctrl + C behaves differently depending on whether anything is selected — the most common source of confusion
- With text selected it copies; with nothing selected it interrupts the running command
- Even a multi-line paste does not run until you press Enter — check the whole thing pasted, then press Enter

4-9. Getting Back After Closing the Window

- **4-9. How to Get Back After Closing the Window**
- Closing the window leaves your files on disk — reopen it, move to the folder, and you are back
- Reassurance: closing the window keeps every folder and file. The only thing lost is the text printed on screen
- 1. Reopen the window — type `pow` in the search box and press Enter
- 2. Move to your working folder — `D:` then `cd foldername` (use Tab)
- 3. Check where you are — when it reads `PS D:\yourfolder>` you are set
- **Common misconceptions**
- ✗ Closing the window destroys what I made → No — the files are still on disk; open Explorer and there they are
- ✗ A new window opens in the folder I was in → No — it always starts at `C:\Users\yourname`; `cd back`
- ✗ If a command does not work the computer is broken → No — it is nearly always the wrong folder or a typo. Check the prompt first
- Unsure where you are? `pwd` — start by checking the current path

4-10. Exercise 3 — Deleting Files and Folders

- **4-10. Exercise 3 — Delete a File, Delete a Whole Folder, Clean Up**
- Deletion cannot be undone — check the path before you press Enter
- # 1. Delete a single file → `Remove-Item memo.txt`
- # 2. Check — no output means the folder is empty → `dir`
- # 3. Delete the practice folder entirely (`-Recurse` means contents too) → `cd .. → Remove-Item -Recurse -Force D:\cli-test`
- # 4. Confirm it is gone — `False` means success → `Test-Path D:\cli-test → False`
- In Explorer: select the file and press Delete / "Move this folder to the Recycle Bin?" Yes (Y) No (N)
- The difference: Explorer sends things to the Recycle Bin; `Remove-Item` deletes immediately, bypassing it. Always check the path first

4-11. PowerShell vs. CMD

Task	blue window)	black window)	macOS · Linux
List files	dir or ls	dir (ls unsupported)	ls
Show file contents	cat	type	cat
Create a folder	mkdir	md	mkdir
Delete a file	Remove-Item	del	rm
Delete a folder	Remove-Item -Recurse	rd /s /q	rm -r
Show current location	pwd	cd (typed alone)	pwd
Clear the screen	cls	cls	clear

- CMD — its command set differs from both PowerShell and Linux
- PowerShell (what this course uses) — its commands are nearly the same as macOS and Linux, so learning them once carries over

4-12. Common Errors and Fixes

What you see	Cause	Fix
"Cannot find path"	Folder name misspelled or missing	Check with <code>dir</code> , then use Tab completion
""ls' is not recognized"	You typed a PowerShell command in CMD	Reopen via <code>pow</code> in the search box
"Cannot run scripts"	Execution policy is blocking it (when using <code>npx</code>)	<code>Set-ExecutionPolicy RemoteSigned</code>
""claude' is not recognized"	You did not restart the terminal after installing	Close the terminal and open a new one
"Access is denied"	Another program is using the folder	Close VS Code and Explorer, then retry

Checkpoint

- 1. I can switch drives with `D:`
- 2. I can create a folder with `mkdir` and enter it with `cd`
- 3. I can list with `dir` and read a file with `cat`
- 4. I can delete files and folders with `Remove-Item`
- 5. I can use Tab completion and the up arrow

Chapter 5. Installing Claude Code

- **Installing, Authenticating and Budgeting Claude Code**
- Install, authenticate with an API key, set up costs, slash commands, CLAUDE.md, permissions and hooks
- What you will learn
 - 1. Running `claude --version`
 - 2. API key authentication and your first conversation
 - 3. Controlling cost through model and effort

5-1. Claude Code Overview

- A terminal-based AI coding agent — why it has taken over
- 1. A terminal-based AI coding agent built by Anthropic
- 2. Instruct in natural language; it creates, edits and runs files on its own
- 3. Five built-in tools — Read / Write / Edit / Bash / Glob, Grep
- 4. CLAUDE.md — project memory, read automatically at the start of every session
- **Six core strengths of Claude Code**
- Skills — bundle design, documentation and workflows into reusable skills
- Plugins — connect MCP servers to integrate external tools (databases, APIs) directly
- Subagents — branch work, run in parallel, isolate context
- Hooks — run automatically before and after work, and enforce hard rules
- CLAUDE.md — persists project context, enabling senior-level collaboration
- Terminal — no IDE dependency; works on Mac, Windows and Linux alike

5-1-1. Harness vs. Model

- **5-1-1. Telling Claude Code (the Harness) Apart from Claude (the Model)**
- Keep the two apart and you can see what `/model` actually changes and where the cost comes from
- **The model — does the thinking**
- Opus 5 / Sonnet 5 / Haiku 4.5 / Fable 5
- Generates prose and code
- Cannot read files itself
- Cannot run commands itself
- You are billed for the tokens it uses
- Typing `/model sonnet` swaps the model
- **The harness — does the doing**
- Claude Code (the terminal program)
- Connects tools to the model (Read/Write/Bash)
- Reads files and passes them to the model
- Actually writes the files the model asks for
- The program itself is free — no charge
- Change the model and Claude Code stays the same
- **Combinations of the same model with different tools**
- Claude Code + Sonnet — what this course uses
- Cursor + Sonnet — same model, different harness
- Claude Code + Opus — same harness, different model
- Calling the API directly + Sonnet — the model with no harness
- In short: you are billed only for the tokens the model reads and writes — drop to Sonnet and set effort to low and the cost falls

5-1-2. Three Coding Agents Compared

Aspect	Claude Code	Codex CLI	Gemini CLI
Maker	Anthropic	OpenAI	Google
Models used	Claude Opus · Sonnet	the GPT family	the Gemini family
Share of professional use	39%	14%	9%
Strength	Pushes long tasks through to the end	Included with a ChatGPT subscription	A generous free tier

- Three tools of the same shape, all running in the terminal — one each from Anthropic, OpenAI and Google
- What they share: all three take sentences in the terminal, so you never have to look at code. This course uses Claude Code, the most widely used of them

5-1-3. How It Differs from Editor-Based Tools

Aspect	Claude Code	Cursor	GitHub Copilot
Form	An agent running in the terminal	A code editor with AI inside	An extension bolted onto your editor
What the human does	States what they want, in sentences	Reads the code and fixes it	It continues what you type
Share of professional use	39%	12%	21%
If you can't read code	Workable	Difficult	Little benefit

- **5-1-3. How It Differs from Editor-Based Tools — Cursor and Copilot**
- Cursor and GitHub Copilot are tools you use while looking at code — a different starting point
- Why this course uses Claude Code: the other two assume someone who can read code — a decisive difference for learners who will not be reading it

5-1-4. What One Line of Instruction Produced

- **5-1-4. What a Single Opening Instruction Produced**
- Built, revised and verified in three sentences — without once opening the code
- > Make me a whack-a-mole game that runs straight in the browser
- ● Write index.html/✓ One file, done – double-click to run
- ♦ Moles pop up at random in a 3×3 grid, with score and time remaining on screen
- > Make it harder – speed it up as time passes
- ● Edit index.html/✓ Applied – the moles come faster as the game goes on
- > Open it in the browser and check it/✓ Scores tally over 30 seconds – works correctly
- What to notice on screen: your own lines start at the far left; ● marks a tool running; ✓ marks a result; and even the fixes are given as sentences, not code

5-2. Prerequisites to Install

Program	Version	Why	Link
Node.js	v24+	Needed to run <code>npx claude-api-setup</code>	https://nodejs.org
Git	2.40+	Version control, GitHub integration, working with Claude Code	https://git-scm.com
Python	3.11+	FastAPI backend development (used in Chapter 10)	https://python.org
VS Code	latest	An editor for viewing and editing files, used alongside Claude Code	https://code.visualstudio.com
Claude Code	2.1+	The core tool of this course; installed on the next slide	https://claude.ai/code

- **5-2. Prerequisites You Must Install**
- All five of these must be installed before the Claude Code exercises

5-2-1. Installing Node.js

- Get the LTS build from nodejs.org — the Windows Installer (.msi) — and leave every component at its default
- LTS vs. Current — LTS is the long-term support stable build. Do not use Current, the bleeding-edge build
- What comes with it — npm and the PATH entry are included by default; nothing extra to select
- The installer — Next, Next, Install, Finish. Change no components
- Verify — in a **new** terminal, run `node -v` and `npm -v`
- Pick the LTS build and click Windows Installer (.msi)
- Verify in a new terminal — a window opened before the install still holds the old PATH and will print nothing

5-2-2. Installing Git

- Get the Windows build from git-scm.com — roughly ten option screens, all left at their defaults, Next each time
- What Git does — saves the state before a change so you can roll back to it when something breaks
- Who uses it — Claude Code, not you. You will never type a Git command in this course
- The installer — about ten option screens; leave all defaults and press Next
- Verify — run `git --version` in a new terminal
- "Click here to download" — get the installer
- Do not change the options — a non-default combination comes back later as an error that is very hard to trace

5-2-3. Installing Python

- python.org — the front-page button has changed, so you need one more click through to the list of installers
- 1. python.org front page — go to the list
- 2. The installer list — Windows installer (64-bit)
- 3. The installer — tick the PATH box, then Install Now
- One checkbox decides everything: **Add python.exe to PATH**. Missing it is the single most common cause of a failed install — if you missed it, uninstalling and reinstalling is faster than fixing it

5-2-4. Install Walkthrough — Windows PowerShell

- Native install via irm, check the version, register the API key permanently, run claude
- # Prerequisite – check Node.js → node --version → v24.15.0
- # 1. Set the execution policy (needed for npx) → Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
→ [Y] Yes [A] Yes to All [N] No: Y

# 2. Claude Code 네이티브 설치 → `irm https://claude.ai/install.ps1`	iex → Claude Code installed successfully`	# 2. Native install of Claude Code → `irm https://claude.ai/install.ps1`	iex → Claude Code installed successfully`
--	--	--	--

- # 3. Check the version → claude --version → Claude Code 2.1.222
- # 4. Register the API key (enter the key the instructor gives you) → npx claude-api-setup
<distributed-key> → API Key configured successfully
- claude → Claude Code started

5-2-5. Install Walkthrough — Windows CMD

- **5-2-5. Install Walkthrough — Windows CMD (Command Prompt)**
- Native install via `curl`, check the version, register the API key permanently, run `claude`
- # Prerequisite – check Node.js → `node --version` → v24.15.0
- # 1. Native install of Claude Code → `curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd`
- # 2. Check the version → `claude --version` → Claude Code 2.1.222
- # 3. Register the API key (enter the key the instructor gives you) → `npx claude-api-setup <distributed-key>`
- `claude` → Claude Code started

5-2-6. Install Walkthrough — Mac Terminal (zsh)

- Native install via `curl`, check the version, register the API key permanently, run `claude`
- # Prerequisite — check `Node.js` → `node --version` → `v24.15.0`

# 1. Claude Code 네이티브 설치 → <code>`curl -fsSL https://claude.ai/install.sh`</code>	bash → Claude Code installed successfully`	# 1. Native install of Claude Code → <code>`curl -fsSL https://claude.ai/install.sh`</code>	bash → Claude Code installed successfully`
---	---	---	---

- # 2. Check the version → `claude --version` → Claude Code 2.1.222
- # 3. Register the API key (enter the key the instructor gives you) → `npx claude-api-setup <distributed-key>` → API Key configured successfully
- `claude` → Claude Code started

5-2-7. When c l a u d e Is Not Recognized

- **5-2-7. When c l a u d e Is Not Recognized — Opening Environment Variables**
- The install did not fail; the window simply does not know where it went. Follow the screens and register it
- 1. Type "environment variables" into the Start menu — Edit the system environment variables
- 2. The Advanced tab — the "Environment Variables" button
- Symptom: c l a u d e --version reports "'claude' is not recognized" — the install finished, but the window does not know where the executable is

5-2-8. Adding the Install Location to PATH

- **5-2-8. Add the Install Location to PATH and Verify in a New Window**
- Add one line to Path under **User variables** — not System variables
- 3. Select Path under User variables — Edit
- 4. New — paste the value — OK
- The value, and what follows: %USERPROFILE%\local\bin. Click OK three times to close out, shut the window, then re-run `claude --version` in a new terminal

5-2-9. A Bulk Installer for the Prerequisites

- **5-2-9. CCKit — a Bulk Installer for the Prerequisites**
- A tool that installs Node.js, Git, Python and Claude Code in one go, and manages sessions too
- What it replaces — everything from 5-2-1 to 5-2-6, done from a single screen
- Where to get it — github.com/hull-kr/Claude-Code-Kit, under Releases, latest version, Assets
- Two files to choose from — try CCKitSetup.exe first; if it is blocked, take CCKitSetup.zip
- GitHub release — download CCKitSetup.exe from Assets
- If you already installed everything individually, skip this. It is for setting up a new machine or a whole lab at once

5-2-10. Getting Past Browser and Windows Blocks

- **5-2-10. CCKit — Clearing the Browser and Windows Blocks**
- The file is unsigned, so the browser and Windows each block it once. This is expected
- Browser block · SmartScreen
- Why it is blocked: a warning triggered by the absence of paid code signing, not a malware verdict. If the .exe is blocked, download the .zip and extract it

5-2-11. Installing CCKit

- **5-2-11. Installing CCKit — No Admin Rights Needed, Prerequisites Included**
- It installs to %LOCALAPPDATA%\CCKit, under your own user account's permissions
- Start the install · The prerequisites are installed from this same screen
- All four are installed together; anything already present is left off the list; an admin-approval dialog may appear while winget downloads

5-2-12. The CCKit Control Panel

- **5-2-12. The CCKit Control Panel — Session List and New Sessions**
- Manages Claude Code sessions per working folder as a list — open and closed states visible at a glance
- Session list — one row per working folder, colour-coded open or closed
- Open button — the same as `c laude - - resume`; picks up where you left off in that folder
- New Session button — pick a folder and a terminal opens with `c laude` already running
- Groups — group by project and your practice folders are easy to find
- Choosing instead of typing — you get the same result from a list without memorizing long options

5-2-13. The `ccd` Shortcut

- **5-2-13. The `ccd` Shortcut — Running Without Permission Prompts**
- It saves you typing a long option every time, but it skips the approval step, so be selective about where you use it
- What `ccd` does — types `c laude --dangerously-skip-permissions` for you
- What gets skipped — it edits files and runs commands without asking
- Where to use it — inside practice folders only. In a folder holding company material, run plain `c laude`
- Tray menu — "Resume" reopens the most recent session as it was
- Be selective — where the approval step matters, run `c laude`, not `ccd`

5-3. Authentication — Registering an API Key

- The credential Claude Code uses to reach Anthropic's servers
- 1. What an API key is — the distributed key your instructor provides; a permit to reach the Claude servers
- 2. How to register it in this course — one line: `npx claude-api-setup <distributed-key>`
- 3. The OAuth route — sign in with your claude.ai account in a browser (if you are using your own paid account)
- 4. Token cost — billed by API usage. This course uses the instructor's distributed key, so it is free for students
- 5. After the course — remove the key with `npx claude-api-setup restore`

5-3-1. How Charges Arise, and How to Cut Them

- **5-3-1. Managing Cost — Where the Charges Come From**
- You are charged for tokens and nothing else — know what inflates them and you can cut the bill through settings
- **How the charge is determined**
- Input tokens — what you typed, plus the files read, plus CLAUDE.md
- Output tokens — the prose and code Claude writes. Priced higher than input
- Context resent — once the conversation is long, every request resends everything so far
- The same question costs more the longer the preceding conversation
- **How to reduce it**
- Model — `/model`. Opus costs more than Sonnet; Sonnet is plenty for the exercises
- Thinking depth — effort. Set it to low on the `/model` screen; higher settings produce more output tokens
- Clear the conversation — `/clear` when a task is done, so earlier content stops trailing along
- Compact it — `/compact` when the conversation is long but must continue; it carries a shortened version of the earlier part
- Turn off MCP tools — merely leaving them on loads their descriptions into every request. Switch off what you do not use
- Check with `/cost` and `/context`
- This course runs on the instructor's key, so students pay nothing. Once you move to your own account, these five items are the difference in your bill

5-3-2. Price per Model, per Million Tokens

Model	Input per MTok	Output per MTok	Cache read	Note
Opus 5	\$5	\$25	\$0.50	Most expensive; for hard judgment calls
Sonnet 5	\$2	\$10	\$0.20	The default for this course
Haiku 4.5	\$1	\$5	\$0.10	Cheapest; simple work
Fable 5.1	\$10	\$50	\$0.25	Specialized for writing

- MTok = one million tokens. Input and output are priced separately, and output is five times the input rate
- Output is 5× — let it answer at length and the bill climbs fast. Ask for only what you need
- Cache reads cost a tenth — resending the same material is far cheaper, and Claude Code uses caching automatically
- Sonnet is enough — at this level of exercise, Opus does not produce noticeably better results
- For the same task, at 50,000 input and 10,000 output tokens: Opus 5 \$0.50 · Sonnet 5 \$0.20 · Haiku 4.5 \$0.10
- Subscriptions are separate — Pro and Max are flat-rate. The table above is pay-as-you-go API pricing
- Source and date — Anthropic's official price list, checked 2026-09-07. Prices change, so check platform.claude.com again on the day

5-4. The Five Built-in Tools

- The tools Claude Code works with
- 1. Read — read a file's contents (analyzing code, checking content)
- 2. Write — create new files (code, configuration)
- 3. Edit — modify an existing file (changing part of the code)
- 4. Bash — run terminal commands (`npm install`, `pip install` and so on)
- 5. Glob, Grep — search files (finding files matching a pattern)

5-5. Slash Commands

Command	What it does	When to use it
<code>/model</code>	Change model and effort together	Right after starting a session: Sonnet + low
<code>/clear</code>	Clear the conversation history	When you switch tasks, to save tokens
<code>/compact</code>	Summarize the conversation to free up context	When it has grown long and slow
<code>/context</code>	Show how much context is in use	When responses start feeling slow
<code>/usage</code>	Check usage and remaining quota (<code>/cost</code> and <code>/stats</code> are aliases)	When you want to know your quota
<code>/init</code>	Generate CLAUDE.md automatically	The first time you open a project
<code>/config</code>	Save settings permanently (language and so on)	When pinning the Korean language setting
<code>/resume</code>	Pick an earlier conversation from a list and continue it	When a session was cut off

- Type `/` mid-conversation to see the available commands
- `/` shows the full list and typing filters it. The ones you will use most: `/model`, `/clear`, `/compact`, `/context`

5-5-1. Create a Practice Folder and Run Claude

- **5-5-1. Creating a Practice Folder and Running Claude**
- Switch to the D drive, create a `claude-test` folder, run `claude`
- # 1. Move to the D drive → `D:`
- # 2. Create the `claude-test` folder and enter it → `mkdir claude-test` → `cd claude-test`
- # 3. Run Claude Code → `claude` → Claude Code v2.1.222 / Sonnet 5, effort low / `D:\claude-test`

5-5-2. Telling the Two Screens Apart

- **5-5-2. Telling the Two Screens Apart — What Changes with Where You Type**
- Every exercise screen from here on is one of these two — tell them apart by the start of the line
- **A. PowerShell — typing commands directly** — PS D:\claudio-test> git init / Initialized empty Git repository /
PS D:\claudio-test> git add .
- **B. Claude Code — instructions in plain language** — > Set up git locally for me. / ● Bash git init ...
- One starts with PS D:\...>; the other starts with a single >
- **Moving between the two**
- PowerShell into Claude Code — `claude` or `claude -c`
- Shell commands without leaving — prefix with `!`, e.g. `!dir`
- Claude Code back to PowerShell — `/exit` (use `claude -c` to come back in)
- The most common mistake: typing plain language into PowerShell, or typing `git init` inside Claude Code. Both fail silently, so check which screen you are in first

5-5-3. What `/init` Does in an Empty Folder

- `/init` analyzes existing code to write `CLAUDE.md`. In an empty folder there is nothing to analyze, so it asks you back
- ✓ The current working directory is empty.
- ✓ There is no codebase to analyze.
- ✓ Please confirm one of the following:
 - ✓ 1. An existing project – tell me the path to analyze
 - ✓ 2. Starting a new project – tell me the type and I will create a template
 - ✓ 3. Working somewhere else – tell me that folder's path
- Do not panic — this is not an error. It is telling you there is no code to analyze because the folder is empty. The wording varies by version
- How to proceed: 1. Choose option 2 and reply · 2. Tell it the project type · 3. A draft `CLAUDE.md` is created · 4. You fill in the content yourself in 5-6
- In an empty folder you can skip `/init` and write it by hand as in 5-6. In an existing project, `/init` analyzes the code and writes it for you

5-5-4. Choosing Model and Effort

- **5-5-4. /model — Choosing Model and Effort on One Screen**
- Pick from the menu, or set them directly with `/model sonnet` and `/effort low`. For the exercises: Sonnet + low
- Select model – Switch between Claude models.
- 1. Default (recommended) – Opus 5 (1M context), Most capable
- 2. Sonnet – Sonnet 5, Everyday tasks ← use this one
- 3. Haiku – Haiku 4.5, Fastest / 4. Fable 5 / 5. Best / opusplan
- Effort (Tab to move here) – low, medium, high, xhigh (default)
- Enter to confirm, Esc to cancel
- Set model to Sonnet 5 for this session only / Set effort level to low (this session only)
- **Keys**
- `/model` — opens this screen
- `↑ / ↓` — move between models (select Sonnet)
- `Tab` — move down to the effort row
- `← / →` — move effort to low
- `Enter` — saves both at once
- Why you must change it: the default is Opus + xhigh — the most expensive model thinking as hard as it can, which multiplies the cost of the exercises. Sonnet + low completes every exercise in this course without trouble
- "this session only" means exactly that — you must set `/model` again each time you start `claude`, or it reverts to Opus

5-5-5. Check These Before Every Session

When	What to type	If you don't
Every time you start a session	<code>/model sonnet</code>	It reverts to Opus and costs climb sharply
Every time you start a session	<code>/effort low</code>	It thinks at xhigh and burns more tokens
Whether you are in the right folder	Check the path at the top of the window	You work in the wrong folder and the results do not line up

5-5-6. Resuming a Session

- **5-5-6. Resuming a Session** — `claude` **vs.** `claude -c`
- For when you closed the window mid-exercise or are moving to the next chapter — get this wrong and the whole prior conversation is gone
- `claude` — starts a new conversation. It cannot see the previous one, so you explain everything again. Only for starting a new project
- `claude -c` — continues the last conversation in that folder exactly where it stopped. This is what you want when coming back mid-exercise, which is most of the time
- `claude -r` — pick from a list. It shows earlier conversations and you choose with the arrow keys. For finding one particular conversation among several
- `PS D:\claude-test> claude -c` → resumes the previous session with the earlier conversation intact
- The most common mistake: typing plain `claude`, opening a new conversation, so the AI knows nothing about the docs/ you built earlier and you start over. Inside a session you can also switch with `/resume`

5-5-7. Picking an Earlier Conversation from a List

- **5-5-7. /resume — Pick an Earlier Conversation from a List and Continue It**
- `claude -c` only gets the most recent one. To choose among several, bring up the list
- Entering from the terminal — PS `D:\claude-test> claude -r`
- Already inside Claude Code — `> /resume`
- Last modified · What was done in that conversation
- 5 minutes ago — wrote the six docs and made the first commit
- 2 hours ago — implemented FastAPI CRUD in backend/
- Yesterday — wrote CLAUDE.md
- Yesterday — terminal command exercises
- Move with `↑` / `↓`, select with Enter
- **Four ways in**
- `claude` — start a new conversation
- `claude -c` — continue the most recent one
- `claude -r` — choose from a list
- `/resume` — switch from inside a session
- Note: conversation lists are stored per folder. Open it elsewhere and you see only that folder's conversations. If the one you want is missing, check where you are first (`pwd`)

5-5-8. Checking Current Usage

- **5-5-8. /context and /cost — Checking Current Usage**
- Real output — the cost rises as the conversation grows
- `> /context` — Model: claude-haiku-4-5 · Tokens: 26.3k / 200k (13%)
- System prompt 6.8k (3.4%) · System tools 18.1k (9.0%) · MCP tools 4.7k (2.3%) · Skills 1.5k (0.7%) · Messages 8 (0.0%) · Free space 173.7k (86.8%)
- `> /cost` — Current session: 16% used · Current week (all models): 27% used
- What's contributing to usage? 88% of usage was at >150k context · Top MCP: playwright 19%
- The point: the bigger the context, the more is resent with every request — so the same question costs several times more in a long conversation. `/clear` when a task ends, `/compact` when it gets long. MCP tools consume context merely by being enabled

5-6. Project Memory

- **5-6. CLAUDE.md — Project Memory**
- The file the AI reads automatically every time
- 1. Lives at the project root — read automatically when c laude starts
- 2. States the tech stack — which language and framework to use
- 3. Coding rules — consistent variable names, function names and comment style
- 4. Prohibitions — "No jQuery; Tailwind only"

5-6-1. Writing CLAUDE.md by Hand

- **5-6-1. Writing CLAUDE.md by Hand (practice — D:\claude-test)**
- Just to see what the file looks like — you are not building an app here. The real project version, with hard rules, is rewritten in 9-1
- Copy the whole thing and paste it straight into Claude Code
- Save the following at the root of this folder as CLAUDE.md.
- `## Practice only – do not actually implement`
- `## Tech stack/- Server: FastAPI + Python 3.11 + SQLite/- Frontend: Vanilla JS + Tailwind CDN/-
Folders: backend/ and frontend/ kept separate`
- `## Coding rules/- Write comments in Korean/- Naming: snake_case in Python, camelCase in JS/- Prefix
every API path with /api/`
- `## Prohibitions/- No jQuery/- No hand-written CSS (Tailwind only)`

5-7. Three Permission Levels and Execution Modes

- 의미 / Meaning
- 자동 실행 – 확인 없이 바로 실행 — Runs automatically, with no confirmation
- 확인 후 실행 – 실행 전 승인 필요 — Requires approval before running
- 금지 – 절대 실행하지 않음 — Forbidden; never runs
- 설명 / Description
- 매번 권한 요청 – 처음 사용 시 권장 — Asks every time; recommended when starting out
- 파일 수정만 자동 승인, 실행 명령은 확인 — Auto-approves file edits, still confirms commands
- 계획만 수립, 실행 전 사람이 검토 후 승인 — Plans only; a human reviews and approves before execution
- 상황을 보고 자동 판단 – 본 과정 기본값 — Judges by situation; the course default
- 묻지 않고 진행 – 익숙해진 뒤에만 — Proceeds without asking; only once you are comfortable
- 권한 확인을 전부 건너뛴 – CLI 로만 지정 — Skips every permission check; settable only from the CLI
- **5-7. The Three-Level Permission System and Execution Modes**
- Claude Code asks permission before running a command — switch modes with Shift+Tab
- Three ways to set the mode: switch mid-session with Shift+Tab; pin it with `/config permissionMode`; specify at launch with `claude --permission-mode`
- A warning on `bypassPermissions`: everything runs unconfirmed, including file deletion. It cannot be set through `/config`, only at launch. Not used in this course — `default` is enough

5-8. /config Settings Explained

Key	Default	Meaning
language	English	Response language; change it to Korean
model	default	Which model; use sonnet for the exercises to save cost
autoCompact	true	Auto-summarizes long conversations; saves tokens, leave it on
thinking	true	Shows the AI's reasoning; useful for learning
checkpoints	true	Saves restore points; recover with Esc Esc
permissionMode	default	Permission mode; default confirms before every action
verbose	false	Prints internal processing too; better left off
theme	auto	Screen theme; dark or light
recap	true	Shows a summary of the previous conversation at session start
autoScroll	true	Follows long output down the screen automatically

- **5-8. /config — the Settings Screen Explained**
- Type /config while Claude is running to see every setting. The ones below are what the exercises need

5-8-1. Recommended Settings

Setting	Value	Why
<code>/config language=한국어</code>	essential, do it first	Claude answers in Korean
<code>/config model=sonnet</code>	sonnet	Can be pinned permanently; this course checks it per session with <code>/model</code>
<code>/config autoCompact=true</code>	true	Auto-summarizes when long; saves tokens
<code>/config thinking=true</code>	true	See the AI's reasoning; useful for learning
<code>/config checkpoints=true</code>	true	Saves restore points; recover with Esc Esc
<code>/config verbose=false</code>	false	Hides internal output; keeps the screen clean
<code>/config theme=dark</code>	dark	If you prefer a dark screen (light also available)

- **5-8-1. Recommended `/config` Settings — for Korean-Speaking Students**
- `/config` is saved permanently after one pass (`/model` must be set again each session)

5-9. The Permissions File, `.claude/settings.json`

- It is normal for the file not to exist right after installation. It is created the moment you answer a permission prompt, on the next slide
- # See what is in this folder → `dir` → `-a----` 2026-08-05 17:33 412 CLAUDE.md
- # To include hidden folders, use `-Force` → `dir -Force`
- # No `.claude` folder yet – this is normal → `Test-Path .claude\settings.json` → `False`
- Its absence is normal — `.claude/settings.json` is created the first time Claude Code asks for permission
- What it holds: the list of allowed commands, the allowed folder paths, the default permission mode — stored per project
- To check once it exists: `cat .claude/settings.json`
- `.claude` starts with a dot, so it is hidden — you must turn on hidden items in Explorer to see it

5-9-1. The Permission Prompt That Creates settings.json

- **5-9-1. The Permission Prompt — This Is When settings.json Is Created**
- All of this happens inside Claude Code. It asks once, right before creating a file or running a command
- 1. You instruct it in plain language — > Make me a docs folder
- 2. The command Claude wants to run — ● Bash(mkdir D:\claude-test\docs)
- 3. It asks just before running — Do you want to proceed?
- 1. Yes / 2. Yes, and always allow access to claude-test from this project / 3. No
- Move with the arrow keys, select with Enter. For the exercises, choose option 2
- **What the three options mean:** 1 — allow this once; 2 — allow this folder from now on; 3 — do not run it
- Choosing 2 creates .claude/settings.json at that moment
- Choose 2 → .claude/settings.json is created → commands in that folder are not questioned again

5-9-2. Three Settings Files, Merged

- **5-9-2. Three Settings Files — Overlaps Are Merged**
- One never erases another. Precedence only comes into play when two values directly conflict
- The files read, top to bottom
- `~/.claude/settings.json` — applies across your whole machine
- `<project>/.claude/settings.json` — shared with the team; committed to the repository
- `<project>/.claude/settings.local.json` — your own values; not committed
- The scope narrows as you go down; on a conflict the lower one wins
- They merge rather than overwrite — set something globally and in the project and both apply; neither erases the other
- Precedence on conflict — for the same key, local beats project beats global
- "Project" means the folder you opened the session in — a `.claude` in a subfolder beneath it is not read. Where you opened it is all that counts
- Hooks are read once at startup — editing them does not affect the current session; open a new one
- Remember: the inner does not beat the outer, it merges with it. But a `.claude` outside the folder you opened is never seen at all

5-10. Eight Practical Tips

- **5-10. Eight Practical Claude Code Tips**

- The essentials worth learning — they speed up everything
- 1. Shell commands without leaving — `> !dir`. Prefix with an exclamation mark to run a shell command from inside Claude Code
- 2. See the plan first — Shift + Tab. In plan mode it shows you the plan before acting
- 3. Reference files with @ — `@CLAUDE.md`, `@backend/main.py`. No full paths needed
- 4. Checkpoints — press Esc twice or type `/rewind` to restore an earlier state
- 5. Paste long instructions — Ctrl + V. Hunt-and-peck typing means typos change the result; copy from the handout instead
- 6. Answer when the AI asks — options 1 / 2 / 3. When your instruction is vague it offers choices; picking one stops it guessing
- 7. When you forget a command — `/help`; Ctrl+C to interrupt. Nothing needs memorizing
- 8. When the result looks wrong — "Do it again, and tell me why it happened." Do not just ask for a fix; ask why it went that way

5-11. Subagents

- **5-11. Subagents — Helper AIs You Hand Work To**

- Leave the main conversation as it is and hand a whole task to a helper AI with its own conversation
- You — instruct the main Claude
- Main Claude — delegates the task to a subagent
- Subagent — carries it out alone in its own conversation
- Main Claude — receives only a summary and reports back

- **When it helps**

- Context isolation — the main conversation stays clean
- Hand off long research and search jobs wholesale
- Fixed roles — one for documents, one for reviews
- Several jobs run in parallel

- **Caution — cost**

- A subagent is a separate conversation and spends its own tokens
- Run several at once and the cost multiplies accordingly
- Measured — sessions that used many subagents accounted for 23% of total usage (per /usage)
- How it differs from the main conversation: the main Claude remembers everything so far; a subagent receives only what that task needs and disappears when done — which is what keeps the main one light
- Not used in this course's exercises — learn the concept and apply it to large research jobs

5-12. Creating and Using a Subagent

- **5-12. Creating a Subagent and Putting It to Work**

- Two ways to make one — ask in plain language, or write the file yourself
- **Method 1 — ask in plain language (the easy way)**
- > Create a documentation agent called doc-writer. → ✓ Created `.claude/agents/doc-writer.md`
- **Method 2 — write the file yourself** — `.claude/agents/doc-writer.md`
- name: doc-writer / description: Dedicated to writing documentation – READMEs and the like / tools: Read, Write, Edit, Glob, Grep / You write technical documentation. Write in Korean.
- Once made — call it by name, or let it be delegated to automatically when the description matches
- > Use doc-writer to write this project's README / > Write a usage document for this project
- Stored at `.claude/agents/` — scope: this project only — for project-specific roles
- Stored at `~/ .claude/agents/` — scope: every project — for roles you use everywhere

Checkpoint

- 1. Run `claude --version`
- 2. Authenticate with the API key and have a first conversation
- 3. Set the model to sonnet and effort to low
- 4. Confirm CLAUDE.md was created

Chapter 6. Skills and Plugins

- **Four Ways to Extend Claude Code**
- Telling plugins, skills, commands and hooks apart, and installing them
- What you will learn
 - 1. The difference between a plugin and a skill
 - 2. Distinguishing commands, skills and hooks by who invokes them
 - 3. Installing a skill from a marketplace
 - 4. Seeing a skill engage without being named

6-1. What a Plugin Is

- **6-1. What a Plugin Is — a Box That Brings In a Whole Bundle of Features**
- Before the how, the what — and how it looks on screen
- `> /plugin → ✓ Plugin marketplaces / ✓ anthropics/skills – 15 plugins / ✓ hull-kr/claude-hud – 1 plugin / ✓ Installed: document-skills · claude-hud`
- Definition — an extension package that brings a bundle of new capabilities in at once
- What is in the box — commands, skills, hooks, agents and external integration settings
- Two ways to install — `claude plugin` from the shell, `/plugin` inside the Claude Code window
- Scope — set scope to project and it is enabled only in that folder
- Nothing to edit by hand — what you enable is recorded in `settings.json` automatically
- Installed as a bundle — bring in the box and its commands and skills all attach at once, installed by the single name registered in the marketplace

6-2. What a Skill Is

- 내용 / Detail
- 작업 절차를 적어 둔 지시서 — A set of written instructions describing a procedure
- `.claude/skills/이름/SKILL.md` — `.claude/skills/<name>/SKILL.md`
- SKILL.md 와 스크립트, 서식 파일을 함께 — SKILL.md alongside scripts and template files
- 이름을 부르지 않아도 요청 문장으로 선택 — Selected from the wording of your request, without being named
- `/스킬이름` 으로 지정해 부르는 것도 가능 — You can also call it directly as `/<skill-name>`
- 플러그인에 포함되거나 폴더 복사로 단독 설치 — Bundled in a plugin, or installed alone by copying the folder
- 그 폴더에서 시작한 세션 — Sessions started in that folder
- **6-2. What a Skill Is — a Single Sheet of Written Procedure**
- One page of human-readable Markdown — one folder is one skill
- `.claude/skills/code-review/SKILL.md` — name: code-review / description: Checks changed code against the review criteria. Use when asked to look at, review or check the code. / allowed-tools: [Read, Grep, Glob]
- `# Code review / ## Review order / 1. Read only the changed files / 2. Find naming-convention violations / 3. Flag missing tests`

6-2-1. What a Marketplace Actually Is

- Not an app-store screen but a public GitHub repository — `anthropics/skills`, the one you type in 6-8, is exactly this
- `github.com/anthropics/skills` — the `.claude-plugin` folder is what marks it as a marketplace
- `skills/` — each folder inside is one skill
- The name you type in the command — `anthropics/skills`, the GitHub account and repository as they are. Not the full URL
- What makes it a marketplace — `.claude-plugin`; without that folder it cannot be registered as one
- The unit of installation — `document-skills`; you pick one bundle inside, not the whole repository
- What one skill really is — a folder plus `SKILL.md`, the single instruction sheet from 6-2. Nothing exotic
- Why a repository: anyone can open their own as a marketplace. This is how you bundle in-house templates as skills and distribute them to a team

6-3. The Three Things Inside a Plugin

Type	Where the file goes	How it engages	In this course
Command	<code>.claude/commands/이름.md</code>	A person invokes it with /name	We only look at the file structure
Skill	<code>.claude/skills/이름/SKILL.md</code>	It selects itself from the wording of the request	We install and actually use one
Hook	<code>.claude/settings.json</code> 의 hooks	Runs automatically at a defined moment	Introduced as a component

- They are distinguished not by what they do but by who invokes them, and when
- Command — only when a person calls it. If you do not call it, it never runs; it never engages by itself
- Skill — the AI does it. When your request overlaps the skill's description it attaches itself, and you can still call it by name
- Hook — the tool, at fixed moments. Executed by the tool itself, not the model: just before and after a tool runs, and at session start and end
- Skills are the superset: a skill can also be called by name, but a command never engages on its own — so most things built for real work are skills

6-4. Command vs. Skill

Aspect	Command	Skill
File location	<code>.claude/commands/이름.md</code>	<code>.claude/skills/이름/SKILL.md</code>
How it is invoked	A person types <code>/name</code>	Claude invokes it from the request; <code>/name</code> also works
Context footprint	The whole body goes into the prompt	Normally one line of description; the body is read only when used
Unit	A single Markdown file	A folder, with scripts, templates and examples
Arguments	Takes a value, e.g. <code>/name backend</code>	Works out the target from the sentence
Distribution	Stays within that project	Can be packaged in a plugin and given to others

6-5. Choosing Between Them

- **6-5. How to Choose Between a Command and a Skill**

- Both live under the skills area, but they engage differently

- **Make it a command when**

- It would be a problem for it to run by itself
- You call it against a different target each time
- It is a bulk cleanup or deployment that is hard to undo
- The rule fits in a line or two
- It should run only when you call it

- **Make it a skill when**

- There is a format to follow and things that are forbidden
- Reports, minutes, official letter formats
- It should apply without being named
- Long rules cost nothing, and it can be distributed
- It decides for itself from the description
- The rule of thumb: if you will call it by name, make it a command; if you want Claude to reach for it on its own, make it a skill

6-6. A Command That Takes an Argument

- 하는 일 / What it does
- 무슨 명령인지, 목록에 뜨는 설명 — What the command is; the description shown in the list
- 뒤에 붙일 수 있는 값을 미리 보여 줌 — Shows in advance which values can follow
- 이 명령이 쓸 수 있는 도구를 제한 — Limits which tools this command may use
- 명령 뒤에 적은 값이 들어오는 곳 — Where the value typed after the command lands
- /테스트 backend 처럼 뒤에 대상을 붙임 — Append the target, e.g. /test backend
- 대상 이름이 파일명에 들어가 따로 남음 — The target's name goes into the filename, so each is kept separately
- **6-6. The Structure of a Command That Takes an Argument**
- The same single Markdown page as a skill — the difference is that you append a target when calling it

description: 지정한 폴더의 테스트를 실행하고 결과를 표로 정리한다 / `argument-hint: [backend \	allowed-tools: Read, Bash, Glob	description: Runs the tests in the given folder and tabulates the results / `argument-hint: [backend \	allowed-tools: Read, Bash, Glob
---	------------------------------------	--	------------------------------------

- # Run tests – target: \$1/1. Run only the tests in the \$1 folder./2. Tabulate the failures./3. Save the result as docs/test-\$1.md.

6-7. Two Routes to Bringing In a Skill

- The difference between pulling in someone else's and putting in your own
- **Marketplace install** — a bundle registered in a public repository, in one command
- Register the repository with `marketplace add`
- Install the bundle with `plugin install`
- The configuration is recorded in `settings.json`
- Several skills arrive at once, and updating is easy
- **Copying the folder** — drop a skill you made straight into the project
- Copy the skill folder into `.claude/skills`
- What to copy — one named folder with its `SKILL.md` inside
- When it is recognized — from the next session started after copying
- Your own formats, absent from any marketplace, work just the same
- Either way the result is the same — it lands under `.claude/skills` and is invoked identically

6-8. Registering a Marketplace and Installing a Skill

- Register the official repository against this project, install the document skills bundle, and inspect what is in it
- # 1. Register the official skills repository for this project → `claude plugin marketplace add anthropics/skills --scope project` → Successfully added marketplace: anthropic-agent-skills
- # 2. Install the document skills bundle → `claude plugin install document-skills@anthropic-agent-skills` → Successfully installed plugin: document-skills (scope: project)
- # 3. Check what is inside the plugin → `claude plugin details document-skills`
- Component inventory / Skills (4) – docx, pdf, pptx, xlsx / Agents (0) · Hooks (0) · MCP servers (0)

6-8-1. Browsing the Marketplace In-Window

- **6-8-1. /plugin — Browsing the Marketplace Inside the Window**
- 6-8 installed from the shell. You can do the same thing by picking from a list inside Claude Code
- `> /plugin → Plugins · Discover · Installed · Marketplaces · Errors · Stats`
- `Discover plugins (1/297) / Search...`
- `frontend-design · claude-plugins-official · 1.2M installs` — a bundle that builds web screens
- `superpowers · claude-plugins-official · 1.1M installs` — brainstorming and subagent workflows
- `code-review · claude-plugins-official · 470.9K installs` — automated PR code review
- `context7 · claude-plugins-official · 442K installs` — an MCP server that fetches up-to-date documentation
- Type to search · Space to toggle · Enter to view · Esc to go back
- From the shell — `claude plugin marketplace add, claude plugin install`. Best when you already know exactly what to type; write it in the handout and copy it
- In the window — `/plugin`. Best when you are looking to see what exists
- Where it is recorded — `.claude/settings.json`; installing adds the name to `enabledPlugins`
- The exercise uses the shell commands from 6-8. Choose in Discover and press Enter; toggle on and off in Installed. All 297 are browsable as a list

6-8-2. Installing claude-hud

- **6-8-2. Installing claude-hud — Install, Then Reload**
- Registering the marketplace works as in 6-8. The setup command is only recognized after the reload
- 1. Register the marketplace — `/plugin marketplace add jarrodwatts/claude-hud`
- 2. Install — `/plugin install claude-hud@claude-hud`
- 3. Reload — `/reload-plugins`
- 4. Configure — `/claude-hud:setup`
- Type them as they are, in order. If it is already installed it tells you
- Skip a step and: calling setup straight after installing reports that the command does not exist. Call it after the reload

6-8-3. What setup Does First — Checking the Environment

- On Windows the shell varies, so it settles the command format first
- Why it checks first — PowerShell syntax is risky here, so it settles on Git Bash form
- `OSTYPE=cygwin` — identified as Git Bash. The command format is decided here
- What you have to do — nothing. It works it out without asking
- Get the shell wrong and the commands fail outright, or the status line renders as a blank row

6-8-3-1. Verifying the Runtime and Commands

- **6-8-3-1. Verifying the Runtime and the Commands**
- Once the format is settled, it actually runs them and checks the output
- What it verifies — the runtime, the dist files, and the statusline command's output
- Install check — version 0.8.0, no ghost or temporary files, node present
- Cache path — under `~/.claude/plugins/cache/claude-hud/`
- No manual configuration — it finds the paths and format itself and writes them into `settings.json`. All you do is choose

6-8-4. If You Already Have a Status Line

- **6-8-4. If You Already Have a Status Line — It Asks Before Replacing It**
- It does not silently overwrite the statusLine you were using
- It asks before replacing — your existing statusLine is not simply overwritten
- Two choices — replace it with claude-hud, or keep yours and quit
- Backup timing — settings.json is copied before you choose
- If you had none, this screen never appears and it moves on

6-8-4-1. Two Backups After Replacement

- **6-8-4-1. After Replacement — Two Backups**
- It keeps both the pre-change settings and the previous command so you can revert
- Pre-change settings — `settings.json.bak.<date>`, a full backup
- The previous command — the original statusline command in `previous-statusline.txt`
- To restore — put the backup back as `settings.json`
- Other settings — permissions, model and theme are left alone; only `statusLine` is replaced
- Reversible — with both backups in place you can return to the original state at any time

6-8-5. Choosing What Else to Display

- Two rows by default. Everything else starts off, and you enable only what you need
- Extra displays — tool activity, agents, session information, session name
- Auto-refresh — 5 seconds, 1 second, or off
- Confirm your choices once more and submit
- For class: turn on all the extra displays and set refresh to 5 seconds — 1 second re-runs the HUD command far more often

6-8-6. The Result — a Status Line Below the Input Box

- **6-8-6. The Result — a Status Line Attached Below the Input Box**
- The screen right after setup finished on the instructor's machine. It took 4 minutes 33 seconds
- Row 1 — model, session name, elapsed time
- Row 2 — context 80%, usage 10%, time until reset
- Row 3 — how many CLAUDE.md files are being read
- Rows 4 and 5 — tools used and how often, and to-do progress (6/6)
- The five rows below the input box are what claude-hud draws
- If you cannot see it, restarting Claude Code reliably fixes it. To change what is shown, run `/claude-hud:configure`

6-9. Using the Installed Skill to Make a Word Document

- **6-9. Making a Word Document with the Installed Skill**

- The Word-document skill attached itself, without a single character of its name being typed
- > Create a new Word minutes template in the docs folder. Five fields: date and time, attendees, agenda, decisions, follow-up actions. Make decisions a table, but leave the cells blank
- ✓ I will use the Word document creation skill
- ● Skill – document-skills:docx
- ● Bash – wrote and ran a script using python-docx
- ✓ Saved docs/minutes-template.docx
- ✓ Paper is A4 portrait with 2.0 cm margins on all sides.
- ✓ Field labels are shaded light grey to distinguish them from the entry cells.
- **Words that were not in the instruction**
- What I never typed — "skill", "docx", "plugin"
- Why it attached anyway — the request overlapped the skill's description
- What actually happened — the Skill tool selected the document skill on its own
- How to confirm — look for Skill in the tool log

6-10. Hooks — Running Your Commands at Set Moments

Event	When	Example use
SessionStart	When a session opens	Report outstanding to-dos
UserPromptSubmit	After you press send	Filter banned words; attach reference material
PreToolUse	Just before a tool runs	Block dangerous commands
PostToolUse	Just after a tool runs	Run a formatter on the edited file
Stop	When it tries to finish answering	Block the end of the turn if work remains
SubagentStop	When a subagent finishes	The same, at subtask level
PreCompact	Just before the conversation is compacted	Write out what should be kept
SessionEnd	When a session closes	Cleanup and logging

- **6-10. Hooks — Running Your Own Commands at Set Moments**
- When Claude Code reaches a particular moment, it runs the command you registered. Eight main events
- Other events: there are more than thirty in total. PreToolUse, UserPromptSubmit and Stop can block execution

6-10-1. Exercise — Installing todo-guard

- The distributed file is `todo-guard.zip` — extract it into the folder below and installation is done, with no commands
- Where to extract it — `%USERPROFILE%\.claude\skills\`. Paste that into Explorer's address bar to go there
- After extraction — `~/.claude/skills/todo-guard/` containing `SKILL.md`, `scripts/check-todo.sh` and `scripts/session-start.sh`
- Scope — placed under `~/.claude/skills` it is available in every project
- Folder name — it must be `todo-guard`; any other name and the skill is not recognized
- Execute permissions — no `chmod` needed on Windows
- Line endings — if execution fails, it extracted with CRLF. Convert the `.sh` files under `scripts` to LF
- After installing: the skill alone does nothing. Hooks must be registered per project

6-10-2. Exercise — Registering Hooks in a Project

- **6-10-2. Exercise — Registering the Hooks in a Project**
- The skill creates `TODO.md`, two hooks and `settings.json` at the project root
- > Set this project up with `todo-guard` → ✓ Created `TODO.md`, two hooks and `settings.json`
- What the skill creates — `TODO.md`, the to-do list, which is the reference point
- `.claude/hooks/todo-check.sh` — the Stop hook; blocks the end of the turn if anything remains
- `.claude/hooks/todo-session-start.sh` — the SessionStart hook; reports what is outstanding when you open
- `.claude/settings.json` — registers the two hooks above
- Where they are created — the folder you ran `claude` in; the skill puts them at the project root
- They do not take effect immediately — hooks are read once when a session opens, so they apply from the next session
- To check it works — try ending a turn with an item unfinished and see whether it is blocked
- To turn it off — delete or empty the hooks entry in `settings.json`
- How to verify — run `todo-check.sh` yourself and check the exit code: 0 if nothing is outstanding, 2 if something is

Checkpoint

- 1. Explain the difference between a plugin and a skill
- 2. Distinguish commands, skills and hooks by who invokes them
- 3. Successfully install a skill from a marketplace
- 4. See a skill engage without being named

Chapter 7. Git and GitHub

- **Git Basics and Connecting to GitHub**
- The basic Git commands, and a successful push to GitHub
- What you will learn
- 1. Checking commits with `git log`
- 2. Pushing successfully to GitHub
- 3. The golden rule — commit before letting the AI work

7-1. Git Overview

- A version control system
- 1. A tool that stores and manages the history of changes to your code
- 2. You can return to an earlier version at any time (rollback)
- 3. GitHub — a service that hosts Git repositories online
- 4. In vibe coding it acts as the safety net

7-2. Why Git Matters More in Vibe Coding

Aspect	Git in conventional development	Git in vibe coding
Size of a change	5 to 20 lines edited by hand	Hundreds to thousands of lines changed from one instruction
Speed	Slow; each line thought through	Fast; whole files can be replaced inside a minute
Blast radius of a mistake	One or two places, traceable by eye	Many files change at once; untraceable
Commit frequency	Incremental commits per feature	Mandatory before every AI instruction; the safety net
Rollback	Occasionally, when needed	Constantly; the moment you dislike the AI's output

- The decisive difference from conventional coding — without Git there is no recovering damaged code
- The conclusion: commit before you instruct the AI. That one commit is what makes damaged code recoverable

7-3. The Core Git Commands

- You can just say it to Claude Code in plain language
- 1. `git init` — initialize the repository. To Claude: "Run git init"
- 2. `git add, commit` — save the changes. To Claude: "Commit everything so far"
- 3. `git push` — upload to GitHub. To Claude: "Push it to GitHub"
- 4. `git log` — check the history. To Claude: "Show me the list of versions"
- 5. `git revert` — roll back. To Claude: "Go back to the previous version"

7-4. The Golden Rule — Commit Before the AI Works

- Always leave yourself a rollback point
- 1. Before instructing the AI → `git commit`
- 2. After the AI changes a lot of code → check the result, then commit
- 3. When something goes wrong → recover with `git revert` or `git reset`
- 4. This habit is vibe coding's safety net

7-5. git init and the First Commit

- **7-5. git init and the First Commit — Two Ways**
- Either gives the same result; this course recommends B. You are inside via `claude -c`, typing in plain language
- **Method A — typing directly into PowerShell**
- # 1. Create the Git repository (default branch main) → `git init -b main`
- # 2. Name and email, applied to this folder only → `git config --local user.name "BulNim" → git config --local user.email "me@example.com"`
- # 3. Commit and check → `git add . → git commit -m "initial commit" → git log --oneline → a3f9c2b initial commit`
- **Method B — in plain language to Claude Code**
- > Set up git locally. Name BulNim, email kimkap10@gmail.com. Then commit.
- ● Bash `git init -b main` / ● Bash `git config --local user.name "BulNim" / git config --local user.email "kimkap10@gmail.com"` / ● Bash `git add . / git commit -m "initial commit"`
- > Show me the commit history → ✓ `git log --oneline` → a3f9c2b initial commit
- The local settings apply to this folder only and touch no other repository on the machine. Saying "set up git locally" is enough for Claude to use `--local`

7-6. Creating a GitHub Repository and Pushing

- **7-6. Creating a GitHub Repository and Pushing — Two Ways**

- Create an empty repository on the GitHub website first, then connect your folder. After that, one sentence does it

- **Method A — typing directly into PowerShell**

- # 1. Create an empty repository on GitHub with "New repository" (leave README unchecked)
- # 2. Connect your folder to GitHub → `git remote add origin https://github.com/BulNim/claude-test.git`
- # 3. Rename the branch to main and push → `git branch -M main` → `git push -u origin main` → * [new branch] main -> main

- **Method B — in plain language to Claude Code**

- > Connect to the GitHub repository `https://github.com/BulNim/claude-test.git`. If it is rejected you may overwrite.
- ● Bash `git remote add origin https://.../` ● Bash `git branch -M main/` ● Bash `git push -u origin main/` ✓ Push complete. You can check it on GitHub
- > Commit everything so far and push
- The first push opens a GitHub sign-in window; sign in through the browser and it will not ask again. If the remote already has a README commit, the push is rejected with "fetch first" — overwrite with `--force` and the remote README is lost

7-7. Files That Must Not Be Uploaded

- **7-7. .gitignore — Files That Must Not Be Uploaded**
- Excluding sensitive and unnecessary files
- 1. .env — API keys, database passwords and other sensitive data
- 2. node_modules/ — npm packages; large, and reinstallable
- 3. __pycache__/ — Python cache files
- 4. *.db — SQLite database files

7-8. Creating and Checking .gitignore

- **7-8. Creating and Checking .gitignore — Two Ways**

- A .env or venv reaching GitHub is a security incident — block it before it goes. You are inside via `claude -c`. If the file is already committed, do this

- **Method A — typing directly into PowerShell**

- # 1. Create .gitignore and fill it → `Set-Content .gitignore ".env" → Add-Content .gitignore "venv/" → Add-Content .gitignore "__pycache__/" → Add-Content .gitignore "node_modules/"`
- # 2. Commit, and untrack .env → `git add .gitignore → git commit -m "chore: add .gitignore" → git rm --cached .env`

- **Method B — in plain language to Claude Code**

- > For the exercise, create a .env file with the single line `API_KEY=dummy` and commit it.
- > Make sure the things that must not be uploaded never get uploaded. → ● Write .gitignore ...
- > Make sure this key is left nowhere in the repository. → ● Bash `git rm --cached .env` → Untracked
- > Check whether this key could still resurface anywhere. → ✓ Not in the latest commit. Still present in earlier commits
- Note: .gitignore only blocks files not yet committed. An already-committed .env must be removed separately with `git rm --cached`. If an API key ever reached GitHub, revoke it and issue a new one

7-8-1. A Key Already Pushed Survives Deletion

- **7-8-1. A Key That Has Already Been Pushed Survives Deletion**

- Untracking it removes it from the latest view — but it remains in earlier commits, and in a public repository anyone can read it
- The latest view — no `.env`, which is exactly where people relax too soon
- An earlier commit — the `API_KEY` is right there
- What it protects against — `.gitignore` only blocks files not yet committed
- What it cannot — anything already committed is merely untracked; it stays in the history
- The illusion of a clean check — ask about the current state and the answer is "it was never uploaded"
- What to do — revoke the key and issue a new one; faster than trying to undo it
- The rule: treat any key that was ever pushed as already leaked. Revoking and reissuing is surer and faster than cleaning history.
The screens above open at a real practice repository — public, so visible without signing in

7-9. Branching Strategy

- This course uses the main branch only
- 1. main — the main branch (the deployment branch, in professional use)
- 2. dev — the development branch (used professionally)
- 3. feature/<name> — one branch per feature (team development)
- 4. This course — main only, to keep things simple

7-10. Final Environment Check

- **7-10. A Final Check of the Whole Environment — Two Ways**

- Check every install and setting in one pass before moving on. One sentence, from inside via `claude -c`. If something is missing, do this

- **Method A — typing directly into PowerShell**

- # 1. Check the installs → `node --version` → v24.15.0
- `git --version` → `git version 2.51.0.windows.1`
- `python --version` → `Python 3.11.9`
- `claude --version` → 2.1.222 (Claude Code)
- # 2. Check Git status and the GitHub connection → `git log --oneline` → a3f9c2b initial commit → `git remote -v`

- **Method B — in plain language to Claude Code**

- > Check this environment. Show me the versions as a table, and confirm whether it can actually push.
- ● Bash `node --version` – v24.15.0 / ● Bash `git --version` – 2.51.0 / ● Bash `python --version` – 3.11.9 / ● Bash `claude --version` – 2.1.222
- ● Bash `git log` – one commit confirmed / ● Bash `git remote -v` – origin connected / ✓ All good. You can move on to the next chapter
- > If anything is missing, tell me how to install it; if nothing is, say so.
- What to check: node v24+, git 2.40+, python 3.11+, claude 2.1+, at least one commit, origin connected. Check separately with `claude update` whether Claude is current. Any one of these failing will stop you in the next chapter, so fix it here

Checkpoint

- 1. Confirm the commit with `git log`
- 2. Push to GitHub successfully
- 3. `.gitignore` includes `.env`
- 4. Run Git commands through Claude Code

Chapter 8. Simple Vibe Practice

- **Simple Vibe Practice — Feeling the Limits First-Hand**
- Build an app from one prompt with no design documents, then analyze the limits
- What you will learn
- 1. Starting in a new folder, `taskflow-simple`
- 2. Generating the app and confirming it actually works
- 3. Discovering four limits for yourself

Three Stages, by How Much Is Specified Up Front

Stage	What is given up front	Folder	Result
Stage 1 (Ch. 8)	Nothing; a one-line instruction	taskflow-simple	The AI picks the stack, database and folders. Different every run; not reproducible
Stage 2 (9-1)	One CLAUDE.md: role, stack, hard rules	taskflow-pro	Stack and folder structure fixed; the docs are reserved by name only, not yet written
Stage 3 (9-2 to Ch. 11)	Six docs: why, what, how, order, conventions	taskflow-pro	Features, order and verification are all fixed. The same result whoever runs it; reproducibility achieved

- **Three Stages of the Exercise — How the Result Changes with How Much Is Given**

- The same app is built three times across Chapters 8 to 11. Only the amount of information given beforehand changes
- c laude - test was a practice folder for learning the tool; no app is built there. App building starts in Chapter 8, across two folders

8-1. What Simple Vibe Means

- Building an app from a single prompt, with no design documents
- 1. No design documents — you start without CLAUDE.md or docs/
- 2. Database, login and folder structure unspecified — the AI decides
- 3. You write no code yourself
- 4. These limits are exactly what motivates the next chapter (CLAUDE.md + docs)

Before You Start — Use a New Empty Folder

- **Before You Start — Work in a New Empty Folder**
- Check these four in order. Get this wrong and every later exercise is affected
- # 1. Create a new folder – not Chapter 5's `claude-test` → `D:` → `mkdir taskflow-simple` → `cd taskflow-simple`
- # 2. Open a new conversation (do not use `-c` to continue) → `claude`
- Inside Claude Code — `> /model sonnet` and `> /effort low`
- The folder you should be in — `D:\taskflow-simple`
- How to confirm — that path printed at the top of the Claude Code screen
- Note: there must be no `CLAUDE.md` and no docs. Work in an existing folder and the earlier exercise's documents get read, changing the result

8-2. Running Claude Code in a New Empty Folder

- Using the Chapter 5 folder changes the result because of its CLAUDE.md — always start in a new folder
- # 1. Create a new empty folder — not Chapter 5's claude-test → `cd .. → mkdir taskflow-simple → cd taskflow-simple`
- # 2. Run Claude Code → `claude` → Claude Code v2.1 · D:\taskflow-simple
- Why a new folder? — `claude-test` holds the CLAUDE.md from Chapter 5, which changes the result. The same applies if a stack is written in your global `~/ .claude/CLAUDE.md`
- Simple vibe — one sentence of natural language, with no stack specified
- Claude's choice — it may pick a different stack each time (Flask, Express, Next.js and so on)
- Why do it this way? — deliberately, so you feel the limits for yourself

8-3. Observing What Claude Produced

- Check which files it created and which stack it chose — it may differ every time
- `> /model sonnet /> /effort low`
- `> Build me a task management app. It needs to add tasks, delete them and change their status.`
- ● Write server.js / ● Write public/index.html / ● Write public/app.js / ● Write public/styles.css / ● Write data/tasks.json
- ✓ I went with Node.js on the backend, plain HTML/CSS/JS on the front, and a JSON file for data
- `> !dir → ✓ server.js · public/ · data/ · .claude/`
- The stack — Claude chose it: Node.js plus plain HTML/CSS/JS
- No database — stored in a JSON file, with limits on concurrency, search and the rest
- No design system — the visual result is decent, but there is no system behind it
- Different every time — in your run it may be Flask, Express or something else

8-4. Running the App and Using It in a Browser

- **8-4. Running the App and Using It in the Browser**
- Confirm it actually works and feel the limits (the port number may differ each time)
- > Actually run the app you just built and tell me the address.
- ● Bash `npm install`/● Bash `node server.js`/✓ Server running on `http://localhost:8000`
- > Check that adding, changing status and deleting all work, and show me the results. Shut the server down when you are done.
- ● Bash `curl -s http://localhost:8000/api/tasks`/✓ Add, change status and delete all worked correctly /✓ Server shut down
- Running the app — Claude runs `node server.js` itself
- How to test — open `localhost:8000` in a browser and click through it yourself
- Confirm it shut down — it may not stop the first time. If the address still loads, tell it to stop again
- Where it could improve — a design system, a database, login, a routing scheme
- Why it matters — you have to see these limits to understand why Chapter 9's `CLAUDE.md` is needed

8-5. The Actual Result — Screen and Folder Structure

- **8-5. The Actual Result — the Screen and the Folder Structure**
- The instructor's real run — the Taskflow app Claude built, and its folder structure
- The browser — localhost:8000
- D:\taskflow-simple\ \ ■■■■ .claude\ ■■■■ settings.local.json / ■■■■ data\ ■■■■ tasks.json ← the store (a JSON file)
- ■■■■ public\ ← the frontend / ■ ■■■■ app.js / ■ ■■■■ index.html / ■ ■■■■ styles.css
- ■■■■ server.js ← the backend (Node)
- # No CLAUDE.md, no docs/, no tests
- # Zero dependencies – plain Node and a JSON file

8-6. Analyzing the Simple Vibe Output

- 한계 / The limits
- 디자인 시스템 부족 – UI 완성도는 양호하나 Tailwind 등 미적용 — No design system; the UI is decent but nothing like Tailwind is applied
- DB 없음 – JSON 파일 저장, 동시 접속·검색 한계 — No database; a JSON file, limited on concurrency and search
- 로그인 없음 – 누구나 접근, 보안 없음 — No login; anyone can get in, no security
- 구조 단순 – 분리는 됐지만 라우터, 체계 없음 — Simplistic structure; separated, but with no router and no scheme
- **8-6. Analyzing What Simple Vibe Produced**
- Both the strengths and the limits are visible — on to the next stage
- The conclusion: simple vibe gets you to prototype level. A real service starts with Chapter 9's CLAUDE.md

8-7. The Cause — Not Enough Context for the AI

- **8-7. What Caused It — the AI Did Not Have Enough Context**
- Why did it turn out this way?
 - 1. You never specified a database → the AI settled on a JSON file, with its concurrency and search limits
 - 2. You never specified authentication → no login, open to anyone
 - 3. You never specified a folder structure → the AI laid it out however it liked
 - 4. No tests, coding rules or prohibitions were specified → solved in the next chapter with CLAUDE.md + docs

8-8. The Root Problem — No Reproducibility

- **8-8. The Root Problem with Simple Vibe — No Reproducibility**
- This is what happens when you do not specify the stack — why pinning it matters (instructor demo)
- Instruction with no stack: "Build me a task management app"
- Attempt 1 — Stack: Node.js + Express · DB: JSON file · UI: Bootstrap
- Attempt 2 — Stack: Flask + Python · DB: SQLite · UI: partly Tailwind
- Attempt 3 — Stack: Django + Python · DB: PostgreSQL · UI: a Vue.js frontend
- > A different result every time
- No reproducibility means no collaboration, which means it never passes senior review
- – The same requirement produces a different structure; results cannot be shared between team members
- – Code review is meaningless, maintenance six months on is impossible, and the code and structure are inconsistent
- > The core value of Chapter 9's document system is exactly this: reproducibility

8-9. Saving the Result, All Through Vibe Coding

- **8-9. git commit and Pushing to GitHub — Saving the Result, All Through Vibe Coding**
- No commands typed by hand — everything in plain language inside Claude Code
- > Set up git locally. Name BulNim, email kimkap10@gmail.com. Create a .gitignore and commit with the message "simple vibe done". Connect it to the GitHub repository <https://github.com/BulNim/taskflow-simple.git>
- ● Write .gitignore/✓ I added node_modules/, *.log and data/
- ● Bash git init -b main / git config --local x2/● Bash git add . / git commit -m "simple vibe done" /
● Bash git remote add origin / git push -u origin main ...
- > Show me the commit history → a3f9c2b simple vibe done
- > Write a guide for this app as README.md and push it. → ● Write README.md → ✓ It covers the app, the stack, installation, running it, its features and the folder structure
- Why push it? — as a record, so the simple-vibe result can sit next to Chapter 9's taskflow-pro for comparison
- All in plain language — no Git commands to memorize; you convey the goal
- Local settings — apply to this folder only; your machine-wide configuration is untouched
- Next — Chapter 9's CLAUDE.md, rebuilding the same app on a designed foundation

8-10. How It Looks on GitHub

- **8-10. How It Looks on GitHub — Code plus README**
- The `taskflow-simple` repository's front page after the commit, push and README in 8-9
- What was pushed — the code, `.gitignore` and `README.md`
- Commits — "simple vibe done", then "docs: add README"
- README — the app, the stack, installation, running it, the API and the folder structure
- Check — the guide renders on the repository's front page
- Next — set it beside Chapter 9's `taskflow-pro` and compare

Checkpoint

- 1. Run the app in a browser and test it yourself
- 2. Discover four limits for yourself
- 3. Understand what caused them
- 4. Commit and push to GitHub

Chapter 9. Systematizing CLAUDE.md and docs

- **Overcoming Chapter 8's Limits — Writing Specs and Systematizing Code Generation**
- This is the stage where the docs get written, so the instructions are long — what you write here becomes the docs verbatim
- What you will learn
 - 1. Starting a new folder, taskflow-pro, with CLAUDE.md first
 - 2. Your specific instruction, then the AI writes
 - 3. Completing CLAUDE.md plus the six documents

Before You Start — Another New Empty Folder

- **Before You Start — What to Type Now (Another New Empty Folder)**
- Check these four in order. Get this wrong and every later exercise is affected
- # 1. Create a new folder – not Chapter 8's taskflow-simple → `cd .. → mkdir taskflow-pro → cd taskflow-pro`
- # 2. Open a new conversation → `claude`
- The folder you should be in — `D:\taskflow-pro`. How to confirm — that path printed at the top of the Claude Code screen
- Note: continuing in the Chapter 8 folder mixes in the simple-vibe result. Chapters 9 to 11 all run in this one folder

Do Not Type the Prompts — Copy and Paste

No.	What it creates	What goes in it	Lines
01	CLAUDE.md	Role, fixed stack, the six doc filenames, six hard rules	17
02	docs/ + 00-overview	Create the six files, a mapping table, reading order 00 to 05	12
03	01-product.md WHY	Goals, personas, MVP scope, out of scope, success criteria	18
04	02-specs.md WHAT	Seven Task fields, five REST APIs under /api/, four screens	21
05	03-design.md HOW	A table of eight technical decisions with alternatives, rationale and trade-offs	15
06	04-tasks.md	Phase 1/2/3 checklists, with a verification method per step	12
07	05-conventions.md	Naming rules, five prohibitions, four implementation rules, and the first git commit	23

- **Do Not Type the Prompts — Copy and Paste Them**
- This chapter's prompts run 12 to 32 lines — typing them by hand introduces typos and changes the result. Chapter 9 creates documents only
- Open the handout — open it in VS Code, preview with Ctrl+Shift+V, and click the copy button at the top right of each code block. The file is 프롬프트_모음_기초편.md (Prompt Collection, Foundations)
- What Chapter 9 creates — only those seven documents. No app code; the app is built in Chapters 10 and 11
- Why ask for an app at all — 9-1-1 is an experiment: type Chapter 8's exact line again and see what one CLAUDE.md changes
- The result — it cannot even start. The stack is fixed, but with nothing saying *what* to build it stops and asks back. Which is why the six docs are needed
- In real work you fill these in by talking with the AI — the docs normally emerge through conversation. They are written out here so every student gets the same result

9-1. Instructions for Writing CLAUDE.md (1 of 2)

- **9-1. Getting Started and Instructing CLAUDE.md (1 of 2)**

- This stage creates CLAUDE.md alone — the six docs are reserved by name; the files come in 9-2
- > In this folder we are building a full-stack web app with a proper structure. Start by writing CLAUDE.md, with exactly these four sections:
 - 1) Your role: a senior full-stack developer of ten years, maintainability first, replies in Korean, identifiers in English
 - 2) Tech stack (fixed – do not change it):
 - - Backend, backend/: FastAPI + Python 3.11 or later + SQLite
 - - Frontend, frontend/: Vanilla JS + Tailwind CDN, two files only – index.html and app.js
 - - Every API path is prefixed /api/; tests use pytest
 - 3) Before starting work: read the six files under docs/ by these names, in this order — 00-overview.md 01-product.md 02-specs.md 03-design.md 04-tasks.md 05-conventions.md (Do not create these six now. Record the names only.)
 - 4) Six hard rules: no guessing / no surprise dependencies / no declaring completion without tests / no hard-coded secrets / no changing the folder structure /
- **From the resulting CLAUDE.md** — ## 1. Your role · - Senior full-stack, ten years / maintainability first · - Replies in Korean / identifiers in English
- ## 3. Hard rules · 1. No guessing – if it is not in the docs, ask · 2. No adding dependencies out of the blue · 3. Never declare something done without tests

9-1. Instructions for Writing CLAUDE.md (2 of 2)

- **9-1. Getting Started and Instructing CLAUDE.md (2 of 2)**
- > If you are given an instruction that conflicts with the docs, ask back before implementing, citing the document and the clause. At this stage, create CLAUDE.md only.
- ✓ Writing... done. All four sections are reflected in CLAUDE.md as specified. / ✓ Ready for your next instruction.
- **From the resulting CLAUDE.md** — 4. No hard-coded secrets · 5. Do not change the folder structure · 6. Do not violate the docs – if something conflicts, ask before implementing

9-1-1. With Only CLAUDE.md — the Same Instruction Again

- **9-1-1. With Only CLAUDE.md — Giving the Same Instruction Again**

- Type Chapter 8's exact line again, and see what is now fixed and what still varies
- > Build me a task management app. It needs to add tasks, delete them and change their status.
- ✓ I have read CLAUDE.md and will follow the fixed stack.
- ● Write backend/main.py – FastAPI + SQLite / ● Write frontend/index.html – Vanilla JS + Tailwind CDN
- ✓ The stack and folder structure are fixed per CLAUDE.md. / ✓ But which fields to include and in what order to build were not in the documents, so I decided on my own.
- **What is now fixed — the effect of CLAUDE.md:** stack, FastAPI + Vanilla JS · folders, backend/ and frontend/ · comments in Korean, identifiers in English · paths prefixed /api/
- **What still varies:** fields — title only, or a due date too? · status — two states or three? · order — backend first or the screen first? · scope — is login in or out? · verification — how far is far enough to call it done?
- **The conclusion:** the stack is fixed, the content is not — which is why the six documents are needed

The Limits of One CLAUDE.md, and the Six Documents

File	Role	OpenSpec equivalent (Advanced)
CLAUDE.md	Entry point, role, routing	(global context)
docs/00-overview.md	Reading order, the whole picture	(table of contents)
docs/01-product.md	Why: goals, personas, scope	> proposal.md
docs/02-specs.md	What: features, APIs, models	> specs/
docs/03-design.md	How: technology, architecture	> design.md
docs/04-tasks.md	Implementation order, checklists	> tasks.md
docs/05-conventions.md	Coding rules, prohibitions, tests	(global conventions)

• The Limits of a Single CLAUDE.md, and the Six Documents

- What a collaborative project needs to pass senior review — and what that looks like split into files

• CLAUDE.md alone — the 9-1 stage

- ✗ Personas, stack, rules and features all tangled in one place
- ✗ When something changes, it is unclear where to look
- ✗ Nothing says what is out of scope, so the AI decides
- ✗ No rationale for trade-offs — zero credibility with a senior
- ✗ No policy on adding dependencies — libraries proliferate

• CLAUDE.md + docs/ — from 9-2 on

- ■ Separation of concerns — why / what / how / order / conventions
- ■ A change means updating just the relevant document
- ■ Out-of-scope is stated, so the AI does not guess
- ■ Alternatives, rationale and trade-offs recorded in a table
- ■ The dependency policy is written down

- The docs you assemble by hand in the Foundations course are generated automatically by one OpenSpec command in the Advanced course

9-2. Creating the Six docs Files and 00-overview

- **9-2. Instructing the Six docs/ Files and 00-overview**

- You specify all six filenames and their contents yourself
- > Create these six files in a docs/ folder, under exactly these names. Add no other files:
- 00-overview.md 01-product.md 02-specs.md 03-design.md 04-tasks.md 05-conventions.md
- Fill in 00-overview.md only. Create the other five as empty files.
- Then check that the doc filenames and order written in CLAUDE.md match the six you just made, and if they differ, correct CLAUDE.md to these six names.
- Contents of 00-overview.md: - One line on the project: TaskFlow Pro – a full-stack web app for team task management - A mapping table of the six files (filename / role / what it tells you) - Reading order: 00 > 01 > 02 > 03 > 04 > 05 - Why it is split this way – explained through separation of concerns
- ✓ 00-overview.md written, with all four items reflected. / ✓ The filenames and order in CLAUDE.md match the six just created. / ✓ Ready for your instructions on 01-product.md.
- **From the resulting docs/00-overview.md** — # 00 – Overview · TaskFlow Pro – a full-stack web app for team task management
- ## Direction of dependency (downward only) · product > specs > design > tasks > conventions · (why) (what) (how) (when) (discipline)
- Decisions upstream become inputs downstream. No reverse influence – a convention must never change the product direction
- ## Reading order · 00 > 01 > 02 > 03 > 04 > 05

9-3. 01-product.md — WHY (1/2)

- **9-3. Instructing 01-product.md — WHY (1 of 2)**

- You specify the goal, the persona, the scope and what is out of scope
- > Write docs/01-product.md only. Do not touch the other docs. With this content:
 - - Goal: make team work visible, so that "who's doing what right now?" stops being asked
 - - Persona: a team lead at a ten-person startup, in their thirties or forties
 - - MVP scope: all four CRUD operations on screen – create, list, update, delete
 - - Status classification plus a due time (due_at), set and displayed – not just a date but a time (e.g. 2026-12-31 18:00)
 - - Light/dark theme toggle (localStorage) / - Mobile responsive at 360px
 - - UI tone: macOS style – rounded corners, soft shadows, translucent cards, system fonts
 - - Future extensions: JWT login, teams, Kanban, chat, CI/CD
 - - Out of scope: external payments, native apps, WebRTC, external calendars, file uploads
- **Excerpt ## MVP scope (all working on screen) — [✓] Four CRUD operations – create, list, update, delete**
 - [✓] Status: todo / in_progress / done
 - [✓] Due time (due_at), date plus time
 - [✓] Light/dark toggle, responsive at 360px
 - [✓] UI tone – macOS style
- **## Success criteria (definition of MVP done) — [✓] Data survives a refresh; nothing breaks at 360px**
 - [✓] API p95 ≤ 200 ms; all four CRUD operations work on screen
 - [✓] Theme toggle persists via localStorage
- WebSocket – the MVP polls; deferred to the extension phase

9-3. 01-product.md — WHY (2/2)

- **9-3. Instructing 01-product.md — WHY (2 of 2)**

- > - Success criteria (only things you can see in the exercise): data survives a refresh / nothing breaks at 360px / all four CRUD operations work on screen / the theme toggle works
- ✓ 01-product.md written. / ✓ Goal, persona, scope, UI tone, theming, out-of-scope and success criteria all reflected.
- **Excerpt — out of scope** ## Out of scope (strictly forbidden) — external payments / native apps / WebRTC / external calendars / file uploads · WebSocket — the MVP polls; deferred to the extension phase

9-4. 02-specs.md — WHAT (1/2)

- **9-4. Instructing 02-specs.md — WHAT (1 of 2)**

- You spell out the data model's fields and all five APIs
- > Write 02-specs.md, fixed to this content:
- - Seven Task model fields, in this order and with these types:
- id (INTEGER, PK, AUTOINCREMENT)/title (VARCHAR 200, required)/description (TEXT, optional)
- status (todo/in_progress/done, default todo)/due_at (DATETIME UTC, optional)/created_at, updated_at (DATETIME, set by the server)
- - Validation: a malformed title, status or due_at → 400; a missing id → 404
- Reject fields not in the spec with 422 – never silently ignore them
- Store due_at in UTC and convert to local time on screen/All three date fields in responses use UTC ISO 8601
- - Five REST APIs, all prefixed /api/: POST 201 / GET list 200 / GET single 200 / PUT 200 / DELETE 204
- **Excerpt** ## Task model fields—id / title* / description / status / due_at (DATETIME UTC, optional) / created_at / updated_at · * = required
- ## Five REST APIs—POST /api/tasks → 201 · GET /api/tasks → 200 (list) · GET /api/tasks/{id} → 200 (single) · PUT /api/tasks/{id} → 200 · DELETE /api/tasks/{id} → 204

9-4. 02-specs.md — WHAT (2/2)

- **9-4. Instructing 02-specs.md — WHAT (2 of 2)**
- > Exclude description from the list response; include it in the single-item response
- - Screen specifications (one table per screen, four in all):
- Create – a form (title / due_at / status)
- List – cards (a status badge plus time remaining until due)
- Update – click a card to open a modal where every field is editable
- Delete – trash icon, confirm, then DELETE
- ✓ 02-specs.md written. / ● The Task model, due_at, validation, the five REST endpoints and the screen specs are all reflected.
- **Excerpt** Create – a form (title / due_at / status) · due_at – a datetime-local picker · List – cards (status badge plus D-N HH:MM) · Update – click a card, then a modal with every field · Delete – trash icon, confirm, then DELETE · Errors: 400 (validation) / 404 (no such id)

9-5. 03-design.md — HOW (1/2)

- **9-5. Instructing 03-design.md — HOW (1 of 2)**

- You specify eight technical and design decisions, and the format of the trade-off table
- > Write 03-design.md as a single eight-row table. Four fixed columns: choice / alternatives / rationale / trade-offs. These eight items, with these numbers and names:
- 1) Backend – FastAPI / alternatives: Django, Express
- 2) Frontend – Vanilla JS + Tailwind CDN / alternatives: React, Vue—the frontend is served by the backend from the same origin; never opened directly via file://
- 3) Database – SQLite (SQLAlchemy ORM, with a later move to PostgreSQL in mind) / alternative: adopt PostgreSQL immediately
- 4) CSS – Tailwind only. styled-components forbidden
- 5) Realtime – the MVP polls every 3 seconds; WebSocket deferred to the extension phase
- 6) State management – module variables and direct DOM updates
- 7) Design system – macOS UI tone / alternatives: Material, Ant
- 8) Theming – light/dark toggle, localStorage, initial value from the system setting
- End with a dependency policy: nothing may be adopted before its rationale is written into this document.
- **Excerpt** ## Design tokens (macOS tone)—Corners – rounded-xl (12px) / 2xl·Shadows – shadow-lg / shadow-xl·Cards – backdrop-blur over a translucent background·Light: bg-white/70·Dark: bg-zinc-900/70·Fonts – -apple-system, BlinkMacSystemFont, ...·Spacing – a 4px grid; touch targets ≥ 44px
- ## Theme toggle—the dark: variant plus localStorage('theme')·initial value from prefers-color-scheme
- ## Absolutely forbidden — [X] styled-components / CSS-in-JS·[X] Redux / Zustand / Recoil

9-5. 03-design.md — HOW (2/2)

- **9-5. Instructing 03-design.md — HOW (2 of 2)**
- > Add httpx to the pre-approved list (needed to run the tests).
- ✓ 03-design.md written – the eight-decision trade-off table, the design tokens, the theme-toggle spec and the dependency policy are all reflected.
- **Excerpt ##** Dependency adoption log – (none yet)

9-5-1. Why Vanilla JS — a Reaction to JavaScript Fatigue

- **9-5-1. Why Choose Vanilla JS — the Backlash Against JavaScript Fatigue**
- Years of framework proliferation, then accumulated developer fatigue, then a return to browser standards
- **2010–2022, framework proliferation**
- React, Vue, Angular — the SPA era. Components and JSX became standard, and the frontend became an "app"
- Frameworks on top of frameworks — Next.js, Nuxt, Remix; SSR, SSG, RSC, a new paradigm every year
- Competing build and state libraries — Webpack to Vite to Turbopack; Redux to Zustand to Jotai...
- The result — six months to a year to learn, bundles over 80 KB, and the very term "JavaScript fatigue"
- **2023 onward, a return to browser standards**
- Web Components stabilized — components built into the browser, adopted by large sites including GitHub
- HTMX grew fast (50k+ GitHub stars) — SPA-grade interaction from HTML attributes alone, widely adopted in business apps
- Hotwire / Alpine.js + Tailwind — DHH's "no-JS framework" stance; effectively the standard for landing pages
- The result — less is more, a return to fundamentals. Welcomed by senior engineers, and a strong fit with the AI era

9-5-2. Real Companies Leading the Shift

- **9-5-2. Real Company Cases — Large Sites Leading the Shift**

- Technical decisions are settled in production, not on Twitter — large companies are stripping SPAs back toward standards
- **GitHub (2023–)** — partially removing React, a gradual retreat. React removed from parts of the PR and Issues pages, moving to server-rendered HTML plus Web Components. "Adopting Web Components cut time-to-interactive dramatically"
- **Shopify (Hydrogen, then simplification)** — adopted Remix, then moved back toward simplicity. It once pushed the React-based Hydrogen hard; operational burden in production grew, and the direction shifted to gradual simplification. "The more traffic you have, the more framework abstraction costs you"
- **DHH (Rails / 37signals)** — declared "We're abandoning TypeScript" in 2023. The founder of the company behind Basecamp and HEY removed TypeScript and released Hotwire under the deliberately provocative banner of a "no-JS framework": "you can build SPA-grade interaction without JavaScript"
- **HTMX — 50k+ GitHub stars** — "an SPA from HTML attributes alone, no JS". Attributes like `hx-get` and `hx-post` give SPA-grade interaction. Adopted by NASA, Walmart and many business apps. "The emblem of the return to vanilla — and job postings mentioning it are rising"
- The common thread: back to fundamentals — choosing the stability of browser standards over the cost of framework abstraction

9-5-3. Why Vanilla in the AI Era

- **9-5-3. Why Choose Vanilla in the AI Era**

- The AI writes the code; the human has to read and verify the result — and legibility is the competitive edge
- **Measured performance**
 - An empty Next.js page — 80 KB+ of JS
 - Vanilla + Tailwind CDN — around 30 KB of CSS
 - JS bundle — zero (no build server)
 - Time to interactive — dramatically faster
- **Signals from the job market**
 - "Strong Vanilla JS preferred" is appearing in senior postings again
 - "Can you debug without the framework?" has become a key evaluation point
 - Understanding JavaScript itself scores higher than "knows React"
 - Senior interviews test the fundamentals: the DOM API, the event loop, fetch
 - The view that reading code matters more in the AI era is spreading
- **Learning curves**
 - The React path — six months. JSX, then components, hooks, state management, the build system, routing. A new abstraction at every step, and a migration with every major version
 - The vanilla path — one week. HTML, CSS, fetch, the DOM — browser standards as they are. No abstraction, and standard APIs that will still work in twenty years
 - The crux in the AI era: the AI writes the code anyway, so the human must be able to read and verify it — and vanilla is overwhelmingly more legible, immediately debuggable by a senior, and a strong fit with vibe coding

9-6. Implementation Order (1 of 2)

- **9-6. Instructing 04-tasks.md — Implementation Order (1 of 2)**

- You specify the phase flow and the checklist format
- > Write 04-tasks.md. The MVP proceeds in three phases:
 - - Phase 1 (design): CLAUDE.md plus the six docs – complete as of now
 - - Phase 2 (backend): backend/ FastAPI → five CRUD APIs → verify in Swagger
 - - Phase 3 (frontend): frontend/ HTML + JS + Tailwind → screens → wire up the API → git push
- Make a checklist table per phase. Fix the step counts: Phase 1 – 10 steps / Phase 2 – 10 steps / Phase 3 – 8 steps
- Three columns: step / how it is verified / done. The done column is a `[]` checkbox, changed to `[x]` as you go.
- Phase 2 dependencies are limited to fastapi, uvicorn, sqlalchemy, pytest and httpx.
- The phase names and count are fixed at those three. No changes.
- From here on, "do the backend" means all of Phase 2, and "do the frontend" means all of Phase 3.
- **Excerpt** ## Progress by phase — Phase 1 (design): write the six docs, 10 steps · Phase 2 (backend): FastAPI CRUD, 10 steps · Phase 3 (frontend): HTML + JS + Tailwind and wiring, 8 steps
- ## Definition of phase completion — a phase is done when every step has passed verification · passing step 3.8 completes the MVP · extensions after the MVP go in a new task document; nothing is appended to this 04-tasks

9-6. Implementation Order (2 of 2)

- **9-6. Instructing 04-tasks.md — Implementation Order (2 of 2)**
- > Rules of progress: in order only, no parallelism, verification required at each step.
- ✓ 04-tasks.md written. / ✓ Checklists for Phases 1 to 3 including verification methods, plus the rules of progress.
- **Excerpt ##** Rules of progress — 1) in order only 2) no parallelism 3) verification required at each step 4) stop on failure

9-7. Conventions and the First Commit (1 of 2)

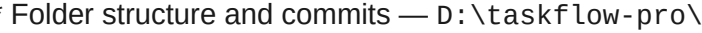
- **9-7. Instructing 05-conventions.md and the First Git Commit (1 of 2)**
- The last document, and the end of Chapter 9
- > Write 05-conventions.md with this content:
- - Naming: snake_case on the backend, camelCase on the frontend, PascalCase for components. Identifiers in English, comments in Korean
- - Five prohibitions, in three columns (prohibited / why / instead):
- print debugging / noise / use the logging module
- bare except / swallows exceptions / catch a specific error
- hard-coded passwords / security incidents / .env plus os.getenv
- the any type in TS / loses all meaning / use explicit types
- !important / tangles specificity / fix the selector instead
- - Put these in .gitignore: __pycache__/, .venv/, *.db, *.log
- - A test matrix (case / request / expected response): Successful create – POST with title only – 201 · List – GET /api/tasks – 200, no description · Single – GET /api/tasks/{id} – 200, with description · Update – PUT with all fields – 200 · Delete – DELETE – 204
- **Excerpt** ## Code-review self-check (always before committing) — [] Did I follow the naming rules? · [] Are all five prohibitions clear (print, bare except, secrets, any, !important)? · [] If I added a dependency, did I write the rationale into 03-design?

9-7. Conventions and the First Commit (2 of 2)

- **9-7. Instructing 05-conventions.md and the First Git Commit (2 of 2)**

- > Missing title – 400 / invalid status – 400 / malformed due_at – 400 / unknown id – 404 / field not in the spec – 422
- - Commit message rules: feat/fix/docs/refactor/test/chore plus a summary in Korean
- When you are done, set up git locally. Name BulNim, email kimkap10@gmail.com. Create a .gitignore and make the first commit: "docs: write the seven Phase 1 design documents"
- Connect it to the GitHub repository <https://github.com/BulNim/taskflow-pro.git>. If it is rejected you may overwrite.
- ✓ 05-conventions.md written. / ✓ git init, add and commit complete. / ✓ Message: "docs: write Phase 1 CLAUDE.md and the six docs". / ✓ What would you like next?
- **Excerpt** [] Is there a matching test, and does it pass? · [] Does the commit message follow the format? · > If any one is unchecked, stop, fix it, and only then report completion
- ## The first git commit — \$ git commit -m "docs: write Phase 1 CLAUDE.md and the six docs"

9-7-1. The Actual Result — Folder Structure and GitHub Push

- The real run — all seven documents written, and the first commit confirmed pushed to GitHub
-  Folder structure and commits — D:\taskflow-pro\
 -  GitHub — BułNim/taskflow-pro/docs

9-8. Chapter 8 vs. Chapter 9

Aspect	simple vibe	systematized docs
Folder layout	<code>server.js</code> + <code>public/</code>	six docs plus CLAUDE.md
Record of decisions	None	Eight decisions with rationale in 03-design
Role definition	None	Stated in CLAUDE.md
Out-of-scope stated	None	Five items listed in 01-product
Change tracking	A single commit	A change-history table at the end of each doc
Consistency of AI output	Different every time	Reproducible, grounded in the docs
Senior review	Throwaway code	A project you can collaborate on

- With the six docs and CLAUDE.md complete, the difference in what comes out
- The point: Chapter 8 gives you only the artifact; Chapter 9 gives you the artifact plus a traceable record of decisions plus reproducibility

Checkpoint

- 1. Create the taskflow-pro folder and write CLAUDE.md
- 2. State the six hard rules in CLAUDE.md
- 3. Write all six documents
- 4. State what is out of scope
- 5. Write the table of technical rationale
- 6. Make the first git commit

Chapter 10. The Backend (FastAPI)

- **"Do the backend" — One Sentence Is Enough**
- Built on Chapter 9's docs — the AI reads 04-tasks and proceeds by itself
- What you will learn
 - 1. Creating backend/, the five CRUD endpoints, and running the server
 - 2. Automatic testing through Swagger UI
 - 3. Seeing CLAUDE.md's hard rules take effect

Before You Start — Two Errors You Will Hit

When it appears	What you see	Fix
Right after running pytest	PermissionError: [WinError 32]	Ask the AI to close the connection in a finally block
Tests pass but there is an error	10 passed	They did pass; only the cleanup failed, for the same reason
Sending Korean through curl	error parsing the body	Save the JSON to a file and use --data-binary "@file"
English works, Korean fails	400 Bad Request	Testing through a browser or Swagger UI avoids it entirely

- **Before You Start — Two Errors That Come Up in the Exercise**

The Payoff of Having the Docs

- **The Payoff of Chapter 9's Docs — One Sentence, "do the backend", Finishes It**
- The AI reads 04-tasks.md and proceeds on its own
- 10-1. "Do the backend" — the AI reads Phase 2 of 04-tasks.md and works through the ten-step checklist
- 10-2. Run the server and Swagger — uvicorn starts automatically and Swagger opens in the browser
- 10-3. Automatic testing of the five APIs — the success case plus 400 and 404; the AI writes the pytest suite too
- 10-4. The hard rules in action — the AI stops before adding a dependency and asks for the adoption log to be updated
- 10-5. Phase 2 complete and committed — all ten checklist items ticked, plus a git commit

10-1. "Do the Backend" — One Sentence Is Enough

- The AI reads Phase 2 of 04-tasks.md and proceeds by itself
- > Do the backend.
- ✓ Phase 2 of 04-tasks.md confirmed.
- ✓ Stack: FastAPI + SQLAlchemy + SQLite (03-design items 1 and 3)
- ✓ Working through the ten-step checklist in order.
- ✓ Written: app/main.py (with CORS) · app/db.py, models.py, schemas.py · app/routers/tasks.py (the five CRUD endpoints) · tests/test_tasks.py (18 pytest cases, per hard rule 3) · requirements.txt and .gitignore
- ✓ Steps 2.1 through 2.10 all verified.
- **The backend/ structure (generated automatically)** — backend/ ■■■ app/ ■ ■■■ main.py ■ ■■■ db.py ■ ■■■ models.py ■ ■■■ schemas.py ■ ■■■ routers/ ■ ■■■ tasks.py ■■■ tests/ ■■■ requirements.txt ■■■ .gitignore
- # All five endpoints implemented/# Including validation, CORS and automatic database creation

10-1-1. The Actual Result — Design, Run, Performance, Phase 2 Verified

- **10-1-1. The Actual Result — Design, Execution, Performance and Phase 2 Verification**
- The real run at a glance — folder design, boot verification, response times, and all of Phase 2 passing
-  The AI's design — the backend/ folder structure (B3/C1)
-  Execution verified — the Phase 2.1 commit and /healthz returning 200 OK
-  Response time — 30 ms at worst, seven times inside the 200 ms target
-  Phase 2 fully verified — all ten steps passing

10-2. Running the Server and Opening Swagger (1 of 2)

- **10-2. Running the Server and Opening Swagger — in Plain Language (1 of 2)**

- You never type a uvicorn command
- > Start the server and open Swagger.
- ✓ Checked Phase 2 of 04-tasks and 03-design.
- ● Bash `cd backend/` ● Bash `uvicorn app.main:app --reload --port 8000`
- ✓ Started in the background at `http://127.0.0.1:8000/` ✓ Swagger UI: `http://127.0.0.1:8000/docs/` ✓ Opening the browser.
- > Looks good
- ✓ Confirmed every endpoint from 02-specs is exposed in Swagger UI: `POST /api/tasks` · `GET /api/tasks` · `GET /api/tasks/{id}` · `PUT /api/tasks/{id}`
- **Swagger UI — API documentation generated automatically** — `POST /api/tasks` Create · `GET /api/tasks` List · `GET /api/tasks/{id}` Read · `PUT /api/tasks/{id}` Update · `DELETE /api/tasks/{id}` Delete · # via the "Try it out" button

10-2. Running the Server and Opening Swagger (2 of 2)

- **10-2. Running the Server and Opening Swagger — in Plain Language (2 of 2)**
- > Looks good / ✓ DELETE /api/tasks/{id}
- # You can call the API directly, without writing any code

10-2-1. The Actual Result — Swagger UI and a POST Call

- **10-2-1. The Actual Result — Opening Swagger UI and Making a POST Call**
- The real run — Swagger UI generated at 127.0.0.1:8000/docs, and a call made directly with "Try it out"
-  Swagger UI at 127.0.0.1:8000/docs, with all five endpoints
-  POST /api/tasks — the request body and Execute

10-3. Verifying the Test Matrix

- **10-3. "10 passed" — Verifying the Test Matrix**

- The ten cases in the 05-conventions matrix, checked against both pytest and the live server
- > Verify against the test matrix in 05-conventions.md.
- ✓ 10 passed – all ten matrix cases pass.
- ✓ Five success cases – create 201 / list 200 / single 200 / update 200 / delete 204
- ✓ Five error cases – missing title 400 / invalid status 400 / malformed due_at 400 / unknown id 404 / field not in spec 422
- ✓ The same ten requests were sent to the running server and the status codes compared – 10 out of 10 match.
- ✓ Isolated with an in-memory SQLite database – tasks.db is untouched.
- **Spec items verified (against 02-specs.md)** — [✓] title constrained to 1-200 characters (empty or over-length both 400) · [✓] Anything outside the three status values rejected (400 on POST, the GET filter and PUT) · [✓] Unknown id consistently 404 (GET single, PUT, DELETE) · [✓] description present on the single item but not the list – the key differentiating contract holds · [✓] updated_at set automatically by the server · [✓] DELETE returns 204 with an empty body · [✓] All validation failures return 400 (FastAPI's default 422 mapped correctly)

10-3-1. The Actual Result — Filling a Gap in PUT V1

- **10-3-1. The Actual Result — Filling the PUT V1 Gap and Satisfying the Matrix in Full**
- The real run — the AI self-checked, noticed the PUT V1 case (empty title) was missing, added the test, and satisfied the matrix completely
-  Spotting the gap — adding the PUT V1 (empty title) case
-  The final matrix — all ten passing, and committed (faa6f95)

10-4. The Hard Rules in Action

- **10-4. The Hard Rules in Action — the AI Stops Before Adding a Dependency**
- A deliberately rule-breaking request. If the AI asks back, it worked; if it just installs the package, CLAUDE.md was set up wrong
- > Add a date-handling library and improve the created_at formatting
- ✓ docs/03-design.md makes no mention of a date library.
- ✓ Prohibition 1 in 05-conventions.md – a package not listed in design.md cannot be added on a whim.
- ✓ This violates hard rule 2 in CLAUDE.md (no surprise dependencies), so I cannot proceed without approval.
- ✓ It can be done with standard features instead – datetime on the backend, Intl and Date on the front.
- ✓ Tell me the format you want and I will implement it without a library.
- > Use the standard library. Make every date format match 02-specs. → ✓ Done using datetime alone. No dependencies added.
- **The core value of the document system — what gets it past senior review:** [✓] The AI does not adopt libraries on its own · [✓] The environment cannot change without the docs changing · [✓] Every dependency has a traceable rationale
- **And if this were simple vibe, with no docs?:** [X] python-dateutil installed automatically · [X] requirements.txt growing unchecked · [X] a dependency jungle six months on
- > This is the moment the document system earns its keep

10-4-1. The Hard Rules in Four Steps

- **10-4-1. The Actual Result — the Hard Rules Demonstrated in Four Steps**
- The real run — propose an improvement, check it against the hard rules, change the code using only the standard library, then verify to the millisecond
-  1) The proposed improvement — a comparison table for `DateTime(timezone=True)`
-  2) Checking the hard rules — a conflict with `CLAUDE.md §3-2`, then options A and B
-  3) The code change — `models.py`, standard library only, zero dependencies
-  4) Verification — `created_at` to the millisecond, before and after

10-5. Backend Complete and Committed

- All ten steps of Phase 2 in 04-tasks.md pass
- > Mark the backend done, commit it and push it.
- ✓ Ticked the Phase 2 checkboxes in 04-tasks.md – 2.1 through 2.10 all complete.
- ✓ Following the commit rules in 05-conventions, starting with feat:``
- \$ git add backend/ docs/04-tasks.md/\$ git commit -m "feat: implement the Phase 2 backend CRUD API"/\$ git push origin main
- ✓ Commit hash a7e3c91/✓ Changes: 18 files, +842 -3 lines/✓ Ready for your Phase 3 frontend instructions.
- **Phase 2 completion check** — ## Phase 2, backend (10 steps, all ticked): 2.1 the backend folder and virtual environment · 2.2 install dependencies (fastapi / uvicorn / sqlalchemy / pytest / httpx) · 2.3 SQLAlchemy models · 2.4 Pydantic schemas with extra=forbid · 2.5 database initialization · 2.6 POST and GET list · 2.7 GET single · 2.8 PUT and DELETE · 2.9 pytest for success, 400 and 404 · 2.10 verify in Swagger
- > The MVP backend is done. Phase 3 can begin.

10-5-1. GitHub Push and Per-Phase Commit History

- **10-5-1. The Actual Result — Pushed to GitHub, with Per-Phase Commit History**
- The real run — `BulNim/taskflow-pro/backend/app`, with the history visible from Phase 2.1 through 2.9 and on to the refactor commit
-  GitHub — `BulNim/taskflow-pro/backend/app` (latest commit 533cafc, a refactor)

Checkpoint

- 1. Create the backend/ folder and the FastAPI skeleton
- 2. Implement all five CRUD endpoints
- 3. Confirm all five work in Swagger UI
- 4. Write and run ten pytest cases — 10 passed
- 5. All ten Phase 2 steps in 04-tasks pass
- 6. Commit to git

Chapter 11. The Frontend (Vanilla JS + Tailwind)

- **The Same Pattern as Chapter 10 — One Sentence, "do the frontend", Finishes It**
- The AI reads the docs and proceeds by itself
- What you will learn
 - 1. Creating frontend/ and wiring up the API
 - 2. Confirming all five endpoints work from the screen
 - 3. Finishing the Foundations course, and pushing to git

The Same Pattern as Chapter 10

- **The Same Pattern as Chapter 10 — "Do the frontend" Finishes It**
- The AI reads Phase 3 of 04-tasks.md (eight steps) and proceeds by itself
- 11-1. "Do the frontend" — the AI reads Phase 3 and works the eight-step checklist
- 11-2. View the screen at the same address — FastAPI serves it via StaticFiles; never open it with `file://`
- 11-3. Test all five endpoints from the screen — POST, GET list, GET single, PUT, DELETE, exercised through the UI
- 11-4. Verify the MVP success criteria — responsive at 360px, data surviving a refresh, API p95 $\leq$ 200 ms
- 11-5. Phase 3 complete and pushed — all eight steps ticked, `git push origin main`

11-1. "Do the Frontend" — One Sentence Is Enough

- The AI reads Phase 3 of 04-tasks.md and proceeds by itself
- > Do the frontend.
- ✓ Phase 3 of 04-tasks.md confirmed. / ✓ Stack: HTML + Vanilla JS + Tailwind CDN / ✓ State: module variables with direct DOM updates (03-design item 6) / ✓ Working through the eight-step checklist in order.
- ✓ Written: frontend/index.html (Tailwind CDN plus the main screen) · frontend/app.js (fetch, DOM manipulation, CRUD) · frontend/style.css (minimal custom additions)
- ✓ Relative fetch(/api/...) paths throughout / ✓ Steps 3.1 through 3.8 all verified.
- **The frontend/ structure (generated automatically)** — frontend/ `index.html` # one line of Tailwind CDN # the main screen (list, input, filters) `app.js` # fetch() and DOM manipulation # the five CRUD functions `style.css` # minimal custom CSS (animations and such)
- # No build step, no separate server / # Reached at `http://127.0.0.1:8000/` / # Responsive at 360px, plus dark mode

11-2. Restarting the Server and Opening the Screen (1 of 2)

- **11-2. Restarting the Server and Opening the Screen — in Plain Language (1 of 2)**

- Now that frontend/ exists, the static-file mount has to be set up again. The AI handles the stop-and-start
- > Shut the server down. → ● Bash find the process using port 8000 → ● Bash kill that process and its children
- ✓ With --reload there are two processes, so I stopped both. / ✓ Port 8000 is free.
- > Start the server again and open the screen. → ● Bash cd backend → ● Bash uvicorn app.main:app --reload --port 8000
- ✓ frontend/ is picked up by the StaticFiles mount (03-design item 2). / ✓ Opening `http://127.0.0.1:8000/`.
- ✓ Main screen: the task list / ✓ Top: the add-task form / ✓ Each task: status toggle, edit, delete
- **Served from the same origin** — # 03-design item 2 – the StaticFiles mount `FRONTEND_DIR = Path(__file__).resolve().parent.parent / "frontend" app.mount("/", StaticFiles(directory=FRONTEND_DIR, html=True))`
- # Browser address: `http://127.0.0.1:8000/` # Never opened directly with file://
- # Tailwind CDN loads automatically: `<script src="cdn.tailwindcss.com">` / # The fetch call in DevTools: `GET /api/tasks → 200 OK, 89 ms`

11-2. Restarting the Server (2 of 2)

- **11-2. Restarting the Server and Opening the Screen — in Plain Language (2 of 2)**
- > Looks good → ✓ Served from the same origin, so fetch returns 200 OK with no CORS involved.
- # Zero build time; changes show on refresh (F5)

11-3. All Five Endpoints, Exercised from the Screen

- **11-3. The Five Endpoints — Exercised Directly from the Screen**

- Not through Swagger but through the real user interface: POST, GET list, GET single, PUT and DELETE
- > Check every endpoint behaves as 02-specs says.
- ✓ POST — add a new task → 201 → reflected immediately ✓
- ✓ GET list — five items shown, no description ✓
- ✓ GET single — the detail modal, description included ✓
- ✓ PUT — the edit modal → 200 → the update shows ✓
- ✓ DELETE — the confirm modal → 204 → removed immediately ✓
- ✓ The list-vs-single description difference from 02-specs is verified on screen.
- **Endpoint verification on screen** — [✓] POST /api/tasks — the add form, reflected immediately · [✓] GET /api/tasks — the list, no description, confirming a light payload · [✓] GET /api/tasks/{id} — the detail modal, with description · [✓] PUT /api/tasks/{id} — the edit modal, all fields updated · [✓] DELETE /api/tasks/{id} — the confirm modal and removal
- **Client-side validation** — [✓] Empty title blocks submission · [✓] Title over 200 characters blocked · [✓] A 4xx response raises a toast

11-4. Verifying the MVP Success Criteria

- The quantitative criteria from 01-product.md — 360px, p95 under 200 ms, data surviving a refresh. Check 360px and the theme toggle yourself, in your own browser
- > Verify the MVP success criteria in 01-product.
- ✓ (1) Data survives a refresh ✓ – SQLite persistence and the fetch calls both correct
- ✓ (2) Responsive at 360px ✓ – no horizontal scroll in DevTools at 360px, no overlapping elements
- ✓ (3) API p95 ≤ 200 ms ✓ – over 50 runs: p50 42 ms, p95 138 ms; every endpoint passes
- ✓ All MVP success criteria met.
- **Measurements** — (1) Data survives a refresh: persisted in SQLite tasks.db; the same list after F5 ✓. (2) Responsive at 360px: DevTools mobile at 360px, no horizontal scroll ✓, no overlap ✓. (3) API p95 ≤ 200 ms over 50 calls: POST 124 ms / GET list 89 ms / GET one 72 ms / PUT 138 ms / DELETE 95 ms

11-5. Phase 3 Complete and Pushed (1 of 2)

- **11-5. Phase 3 Complete and Pushed — the Foundations Course Finished (1 of 2)**

- All eight Phase 3 steps in 04-tasks.md pass
- > Mark Phase 3 done, commit it and push it.
- ● Bash `git add . / git commit / git push origin main`/✓ Commit 4b2e7f1, default branch main confirmed
- > Write a guide for this app as README.md and push it. Make sure it does not contradict the docs.
- ✓ Ticked the Phase 3 checkboxes in 04-tasks – 3.1 through 3.8 all complete.
- ● Bash `git remote -v` – origin connection confirmed
- \$ `git add frontend/ docs/04-tasks.md / $ git commit -m "feat: Phase 3 frontend (HTML+JS+Tailwind) plus API wiring" / $ git push origin main`
- ✓ Commit 4b2e7f1, default branch main confirmed/✓ Changes: 7 files, +482 -3 lines
- **Phase 3, frontend (8 steps, all ticked):** 3.1 the frontend folder and index.html · 3.2 Tailwind CDN and the theme toggle · 3.3 rendering the list cards · 3.4 the add form · 3.5 the edit modal – fetch the single item, then all fields · 3.6 delete confirmation · 3.7 responsive at 360px · 3.8 measuring API p95 against 200 ms

11-5. Phase 3 Complete and Pushed (2 of 2)

- **11-5. Phase 3 Complete and Pushed — the Foundations Course Finished (2 of 2)**
- > Write a guide for this app as README.md and push it. Make sure it does not contradict the docs.
- ✓ The Foundations MVP is complete. You can move on to OpenSpec in the Advanced course.
- > Foundations MVP complete

11-6. The Actual Frontend Result

- **11-6. The Actual Frontend Result — Dark and Light, Verification, GitHub**
- The real run — both theme screens, S1/S2/S3 passing, and the frontend/ structure pushed to GitHub
-  Dark mode — TaskFlow Pro in the macOS tone
-  Light mode — the same UI, with the theme toggle working
-  Success criteria verified — S1, S2 and S3 pass, with all five endpoints measured
-  Pushed to GitHub — backend/, docs/, frontend/ and the README

11-7. Final Verification — A Clean Run from the Fixed Prompts

Check	Result	Evidence
The six doc filenames match CLAUDE.md	Pass	Names match one to one
Update: fetch the single item, then PUT every field	Pass	description is preserved
No description in the list, present on the single item	Pass	The response keys differ as specified
POSTing a field not in the spec returns 422	Pass	extra_forbidden
Ids are not reused after deletion	Pass	Delete 5, the next is 6
pytest	Pass	10 passed

- **11-7. Final Verification — Rerunning from Scratch with the Finalized Prompts**
- Run from an empty folder, 9-1 through 11-5 in order. These are the resulting screens, untouched
- *(image)* Dark mode (default) · Light mode (after toggling) · The edit modal with every field · 360px

Checkpoint

- 1. frontend/ with index.html, app.js and Tailwind CDN
- 2. All five endpoints working from the screen
- 3. Nothing breaks at 360px
- 4. Measured API p95 under 200 ms
- 5. All eight Phase 3 steps in 04-tasks pass
- 6. Pushed with `git push origin main`

Chapter 12. Looking Back, and Looking Ahead

- **Analyzing AI-DLC, Concluding on OpenSpec, and Previewing the Advanced Course**
- Analyzing the results of Chapters 9 to 11, and preparing for the Advanced course
- What you will learn
 - 1. AI-DLC as advertised vs. as it really is
 - 2. AI-DLC's automation is really the docs a human defined
 - 3. Automating that up-front definition with OpenSpec

12-1. What the Foundations Course Covered

- **12-1. What You Learned in the Foundations Course**

- A summary of Chapters 1 to 11
- Ch. 1 AI overview — the concept of AI plus seven example services
- Ch. 2 Using AI in development — planning, assisted coding, vibe coding, shipping a service
- Ch. 3 Vibe coding + AI-DLC — definitions, roles, pitfalls
- Ch. 4 Terminal basics — ten commands plus Tab completion
- Ch. 5 Installing Claude Code, API key authentication, Sonnet with low effort
- Ch. 6 Skills and plugins — installing from a marketplace and invoking them
- Ch. 7 Git and connecting to GitHub
- Ch. 8 Simple vibe practice — feeling the limits
- Ch. 9 CLAUDE.md plus six docs — a reproducible structure
- Ch. 10 A FastAPI CRUD API, tested through Swagger
- Ch. 11 Vanilla JS and Tailwind, front and back wired together

12-2. AI-DLC as Advertised vs. as It Really Is

- 현실 – 9~11장 실습 결과 / The reality, per Chapters 9 to 11
- 인간 사전 투자가 90% – CLAUDE.md, docs 6종, 절대규칙 — 90% is human investment up front: CLAUDE.md, the six docs, the hard rules
- AI는 컨텍스트가 있어야 주도 – 빈손이면 매번 다른 결과 — The AI can only lead with context; empty-handed it gives a different answer every time
- 사전 투자 없이 단계 건너뛰기 불가 – 사전 골격이 모든 자동화의 전제 — You cannot skip steps without the up-front investment; the skeleton is the precondition for all automation
- 인간이 '왜, 무엇, 언제' 먼저 – AI는 '어떻게'만 실행 — The human settles why, what and when first; the AI only executes the how
- **12-2. AI-DLC — the Ideal vs. the Reality**
- The AWS material shows only what is above the waterline — 90% of it is human investment below
- The point: AI-DLC is still a marketing slogan — it does not work without human investment up front

12-3. AI-DLC's Automation Is the Docs a Human Wrote

Stage	As AWS puts it	What the human actually defines	What goes in it
Requirements	"AI analyzes it automatically"	docs/01-product.md	Goal, persona, scope, out of scope
Spec	"It proposes the what and how"	docs/02-specs.md	The Task model, five REST endpoints, validation rules
Code generation	"Code generated automatically"	docs/03-design.md	Stack decisions, trade-offs, dependency policy
Testing	"The AI writes and runs them"	docs/05-conventions.md	The test matrix and the five prohibitions
Deployment	"CI/CD runs itself"	docs/04-tasks + 05-conventions	Phase checklists, deployment conventions, git rules

- **12-3. AI-DLC's Automation Is Really the Docs a Human Defined**
- The marketing language vs. how it actually works — mapped onto the six documents
- The automation across AI-DLC's five stages is the result of what a human defined in those six documents beforehand
- The AWS material shows only the automated result and leaves out the human's document work that precedes it
- AI-DLC is AI-led in name; in practice the human leads through the docs and the AI executes

12-4. Automating the Up-Front Definition

Aspect	Foundations (by hand)	Advanced (OpenSpec)
Human investment up front	Six documents written by hand, every time	The framework supplies the skeleton
Consistency	Everyone's format differs	Standardized as proposal, specs, design, tasks
Change tracking	Manual diffs	Deltas from proposal to specs, automatically
Reusability	The next project starts from nothing	The specs are reusable, whatever the language or stack
How much the AI can lead	Only as far as the up-front work reached	Wider, because the skeleton is already there

- **12-4. OpenSpec — Automating the Up-Front Definition That Completes AI-DLC**
- From hand-written docs in Foundations to OpenSpec automation in Advanced — standardization and reuse
- The marketing story, then the reality that a human must define it first, then automating that with OpenSpec
- Standardizing the preparation into a framework is what finally brings AI-DLC within reach
- > This is the case for the OpenSpec exercises in the Advanced course — the moment a tool takes over the human's up-front investment

12-5. Mapping the Docs to OpenSpec

- 심화편 OpenSpec / OpenSpec equivalent
- 유지 (전역 운영 규칙) — Kept, as the global operating rules
- 유지 또는 README.md — Kept, or becomes README.md
- proposal.md (WHY)
- specs/ 델타 (WHAT) — the specs/ delta (what)
- design.md (HOW)
- tasks.md (실행계획) — the execution plan
- 유지 (전역 규약) — Kept, as the global conventions
- **12-5. Mapping the Foundations Docs to OpenSpec — and the Checkpoints**
- The hand-written Foundations documents mapped to the OpenSpec artifacts of the Advanced course
- **Foundations checkpoints**
- [] Ch. 9 — write CLAUDE.md and the six documents
- [] State the six hard rules and what is out of scope
- [] Ch. 10 — "do the backend", finished in one sentence
- [] 10 passed — the test matrix verified
- [] Ch. 11 — the frontend, wired to the backend
- [] Committed and pushed
- Specific instruction, then AI execution grounded in the docs, then a reproducible result — that is what vibe coding really is